

Chapter 6. Docker & Kubernetes



Lee Youngkon

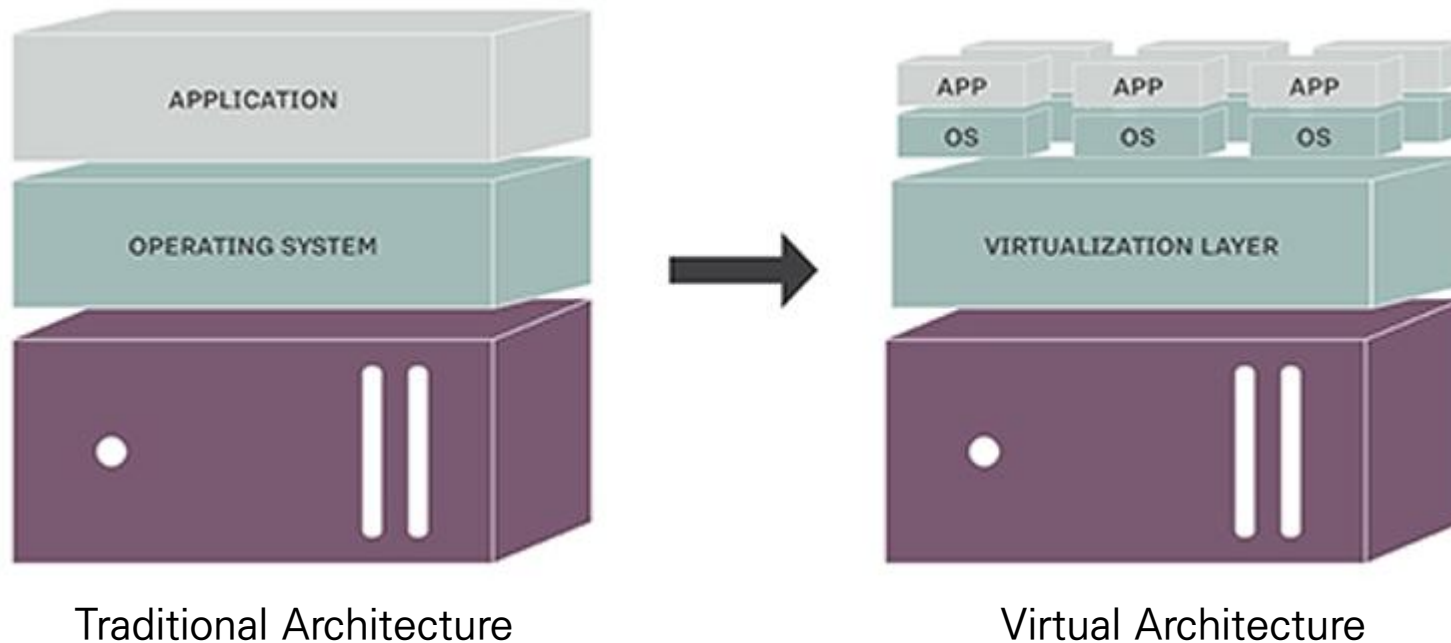


A top-down view of a wooden desk with a laptop, glasses, a cup of coffee, and a small plant. The desk is made of dark wood with visible grain. A silver laptop is in the top right corner. A pair of black-rimmed glasses is in the center. A white cup of black coffee is in the bottom right. A small green succulent is in the top center. A white L-shaped line is on the left side.

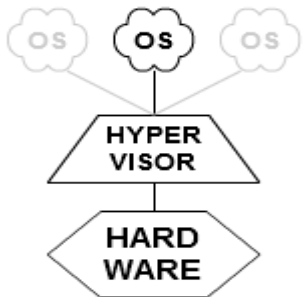
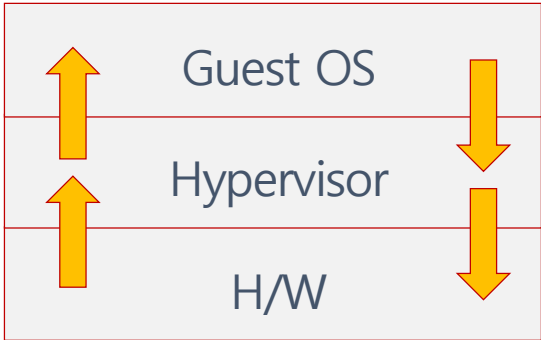
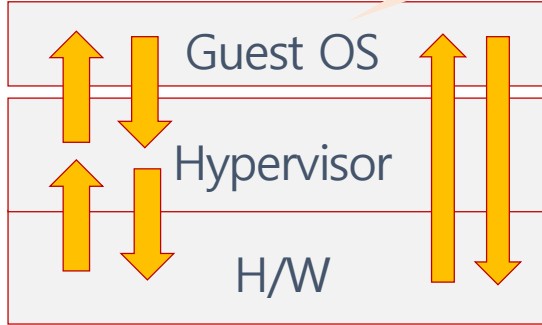
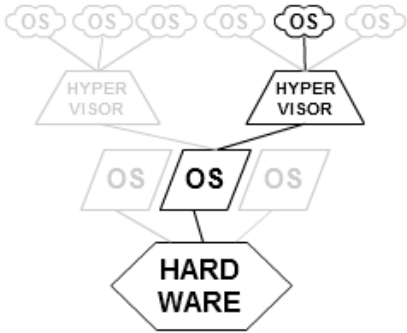
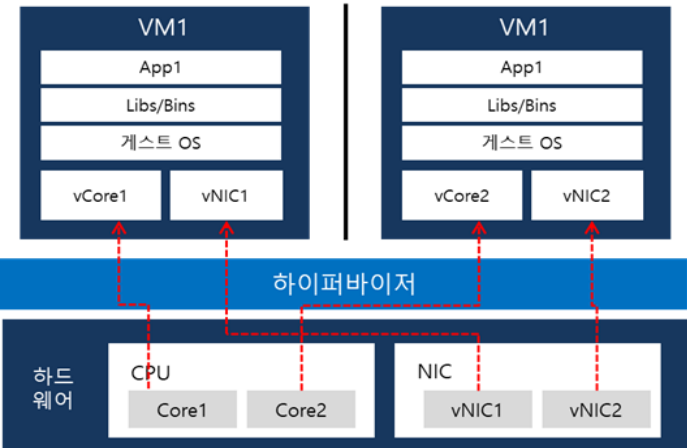
01. Introduction of Container

- ✓ Virtualize physical resources to provide flexible infrastructure

Traditional and Virtual Architecture



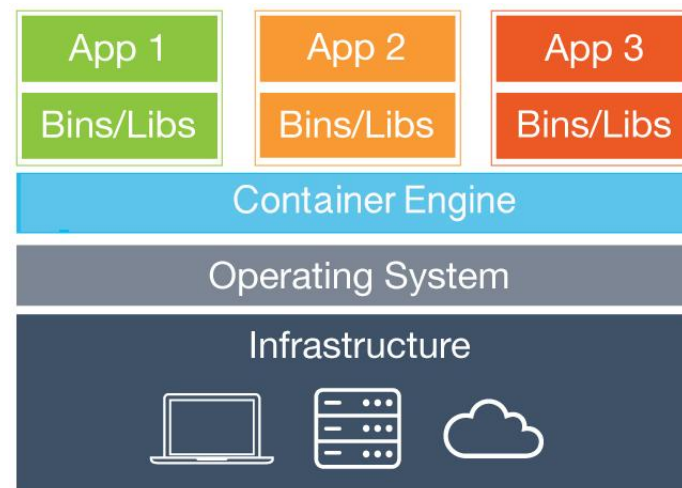
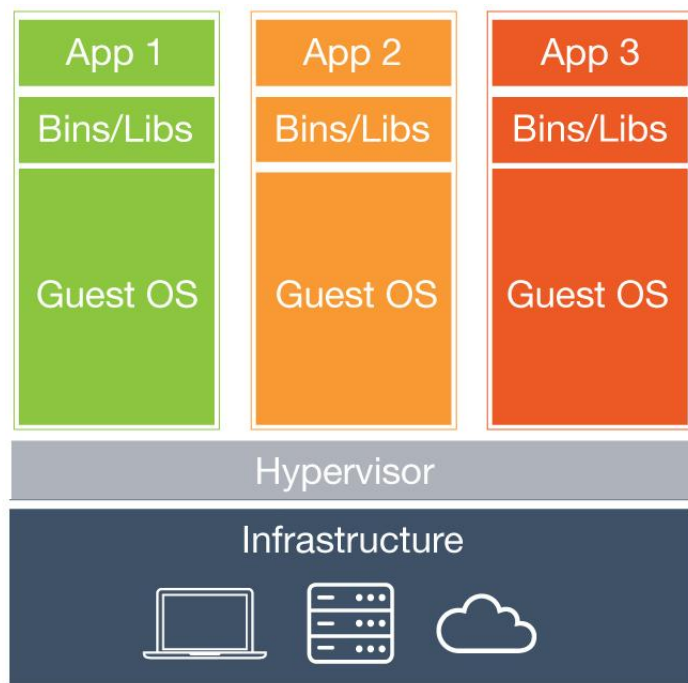
Virtualization Type

	Full virtualization	Half virtualization
Native(Baremetal): Xen, Hyper-V, KVM 		modification 
Host: VMware 	격리 	

Hypervisor

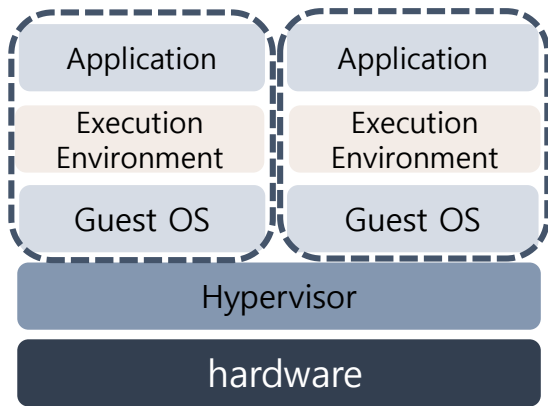
VS

Container



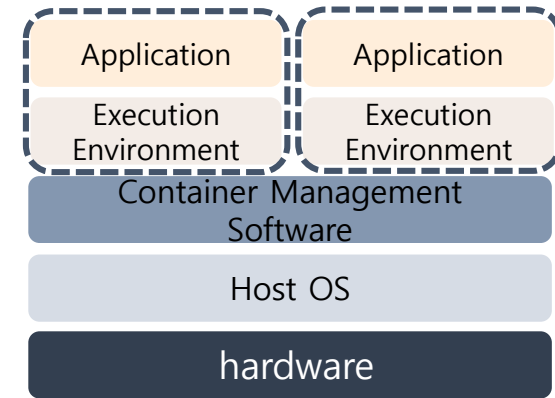
Types of Virtualization

Hypervisor



VS

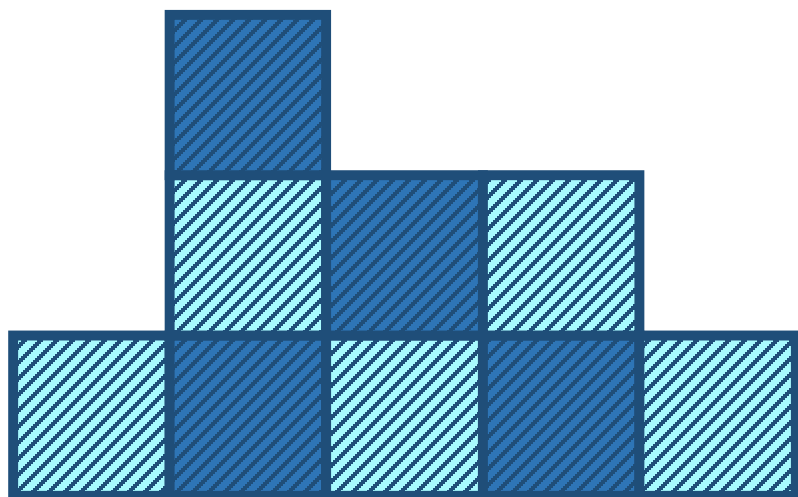
Container



feature	✓ Configuring a Running Environment with Guest OS on Hypervisor	✓ Configure multiple execution environments using container management software on a host OS basis
Virtualization Level	✓ Hardware-level virtualization	✓ OS-level virtualization
Resource Usage	✓ Requires a lot of resources such as OS installation	✓ Use the required resources in the execution environment
Running time	✓ Start time in minutes	✓ Start time in seconds
Security	✓ Fully isolated for each VM, safer than container virtualization	✓ Isolated at process level, less secure than Hypervisor virtualization



What is a container?



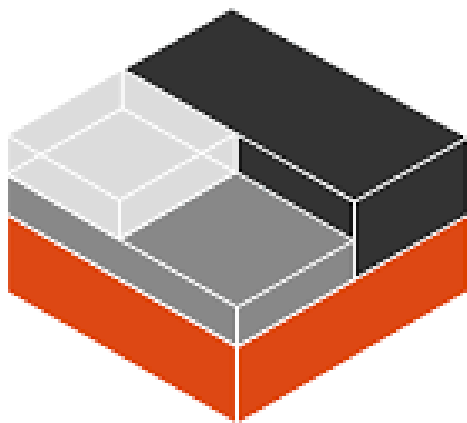
container

OS packaged with minimal elements for a program to operate, a kind of virtual machine that enables independent deployment and execution

Linux Container



What is a Linux
container?



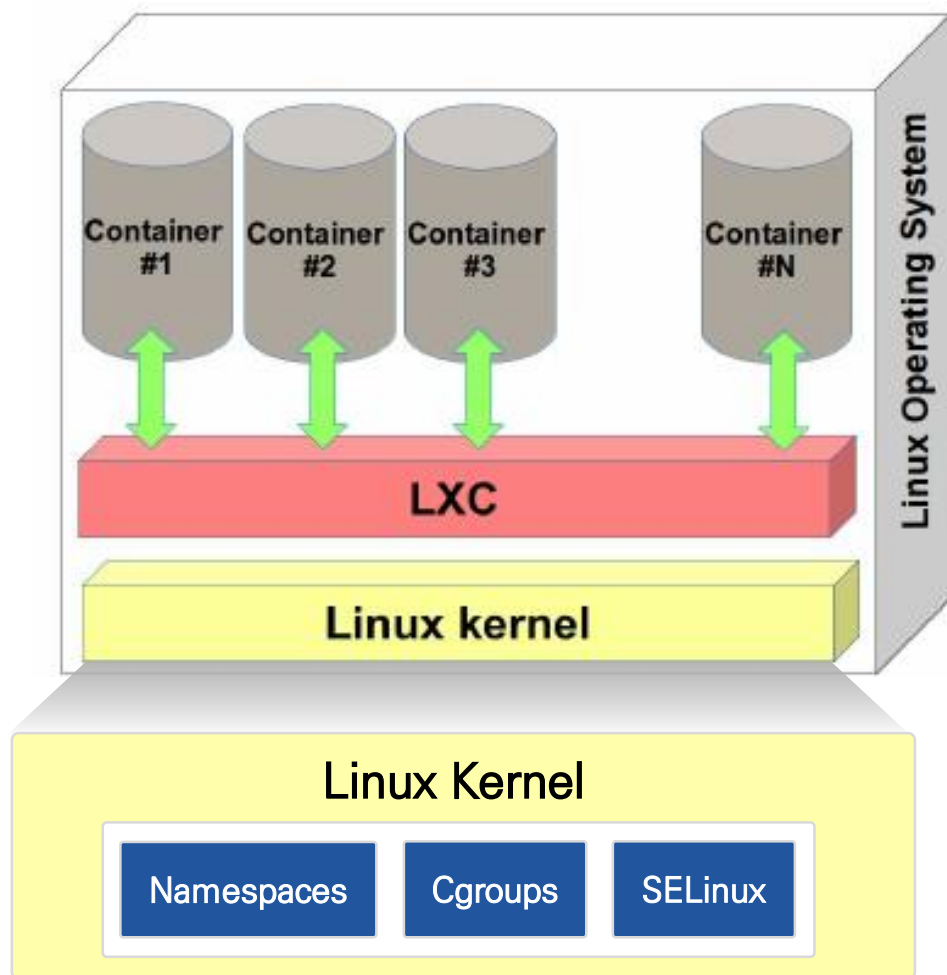
LXC

LXC

Linux Containers (LXC) is an operating system-level virtualization method for running multiple isolated Linux systems (containers) on a single control host

LXC Technology

Source: Grep Haxs



Definition

- ✓ A collection of tools for managing containers
- ✓ User interface for accessing container-related features from the Linux kernel

Characteristics

- ✓ Operating System-Level Virtualization
- ✓ Fast execution and efficiency
- ✓ High portability

Linux Container

Kernel

- ✓ Components that are critical to running containers on the Linux kernel

Cgroups

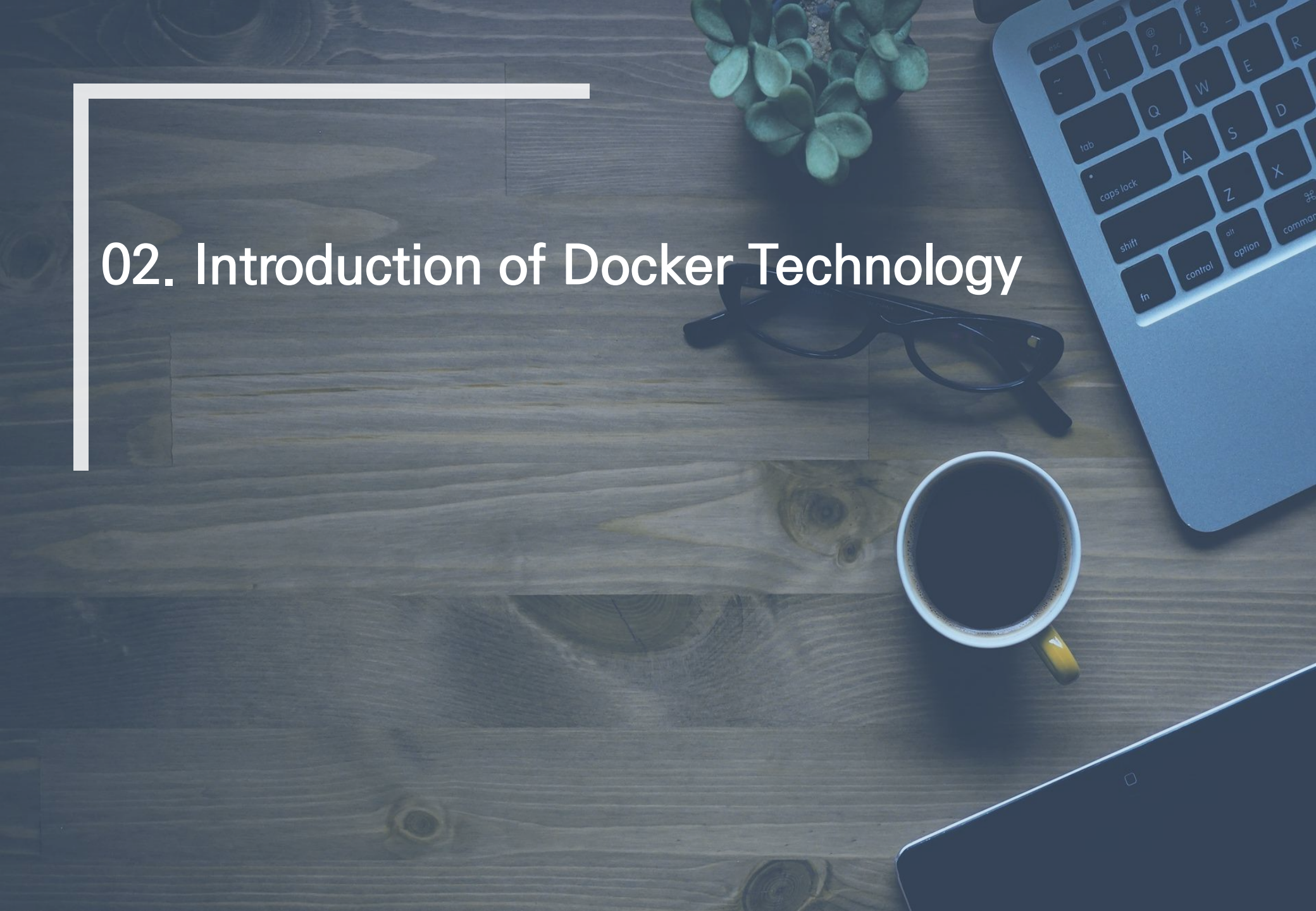
Allocate the necessary resources (CPU, memory, block I/O, network, etc.) to the container

Namespaces

The role of isolating the created container from other containers

SELinux

The role of protecting the kernel by providing kernel-level security

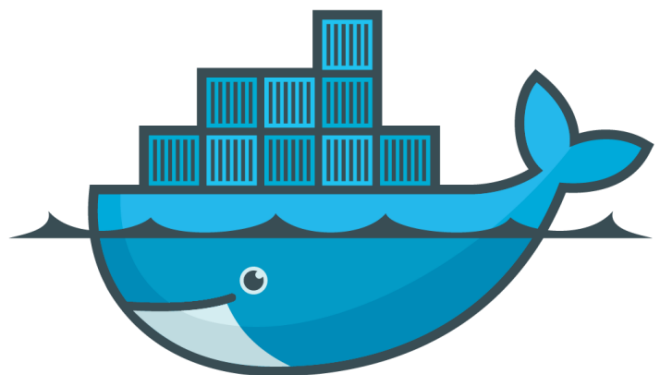
A top-down view of a wooden desk with a laptop, glasses, a cup of coffee, and a small plant. The desk is made of dark wood with visible grain. A silver laptop is in the top right corner, showing the keyboard. A pair of black-rimmed glasses is in the center. A white cup of black coffee is in the bottom right. A small green succulent is in the top center. A white L-shaped line is on the left side of the image.

02. Introduction of Docker Technology

Docker



What is a docker?



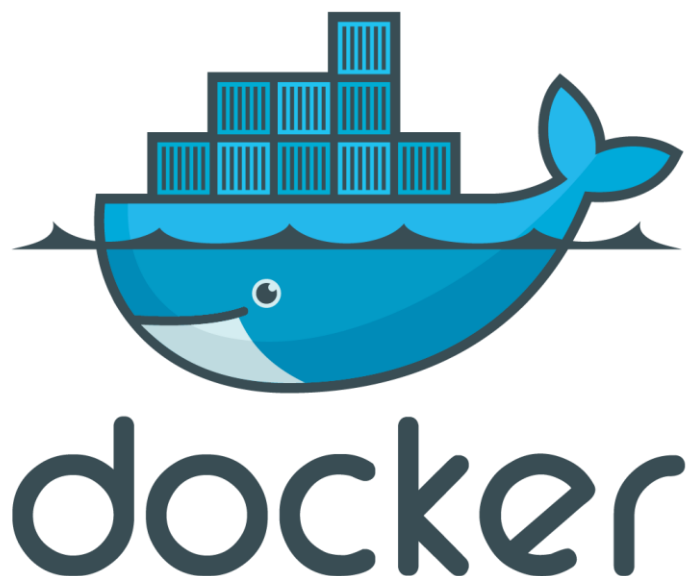
docker

Docker

Open source virtualization platform that manages and operates containers, including applications and execution environments

Docker

Docker Features

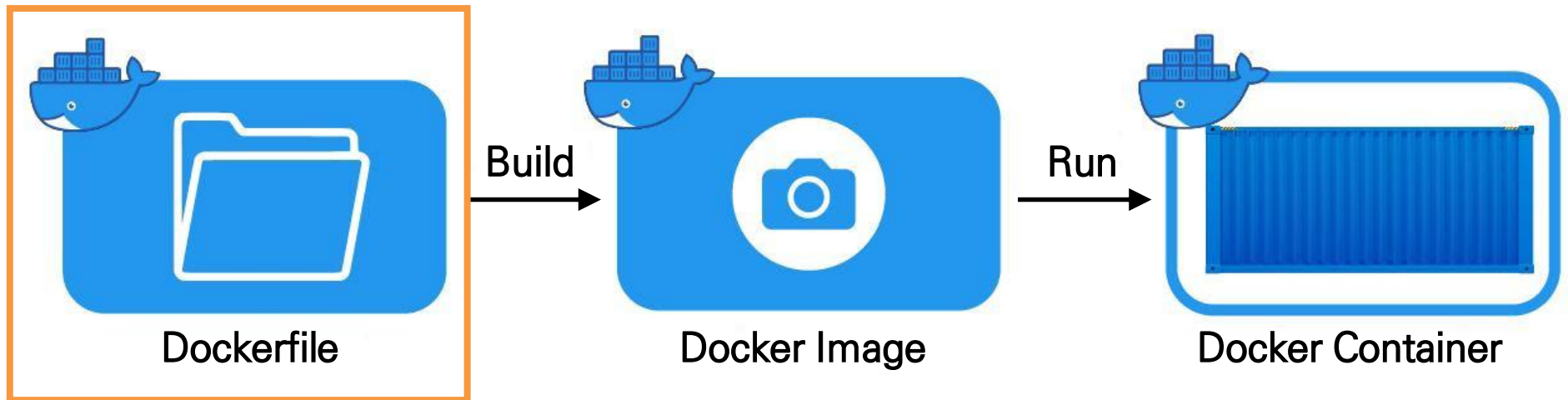


- ✓ Fast execution speed
- ✓ Easy-to-Hand Deployment
- ✓ light capacity
- ✓ Application Independence
- ✓ Ensuring High Security
- ✓ Low overhead
- ✓ Create and share images

Dockerfile

Source: JFrog

Docker files



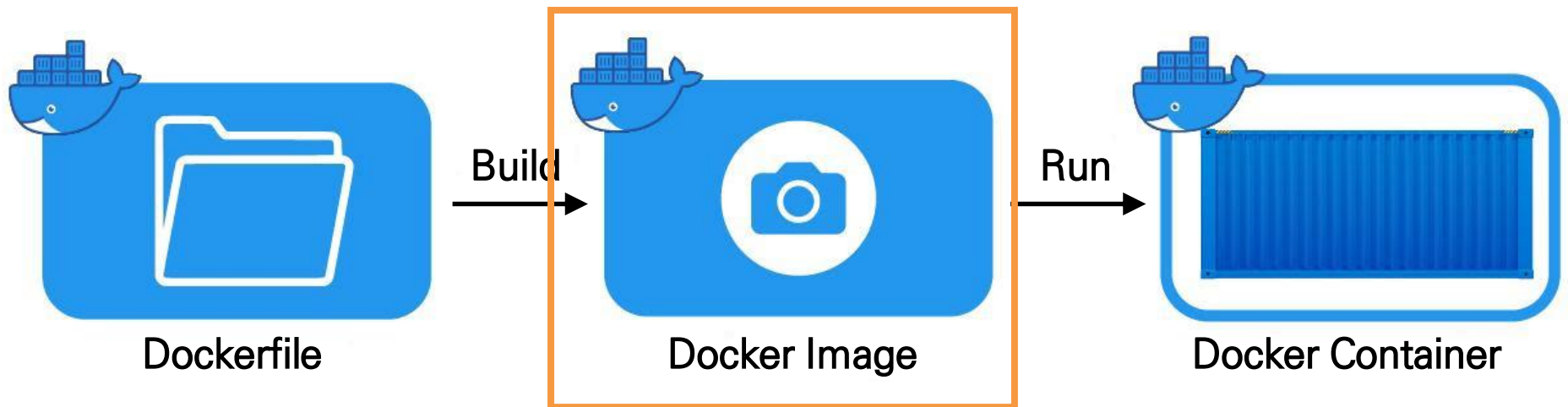
- ✓ Dockerfile Docker files are settings files for creating docker images and are created using various docker file commands

03 Docker Image Definition

Source: JFrog

Docker Image

Docker image



✓ Docker Image

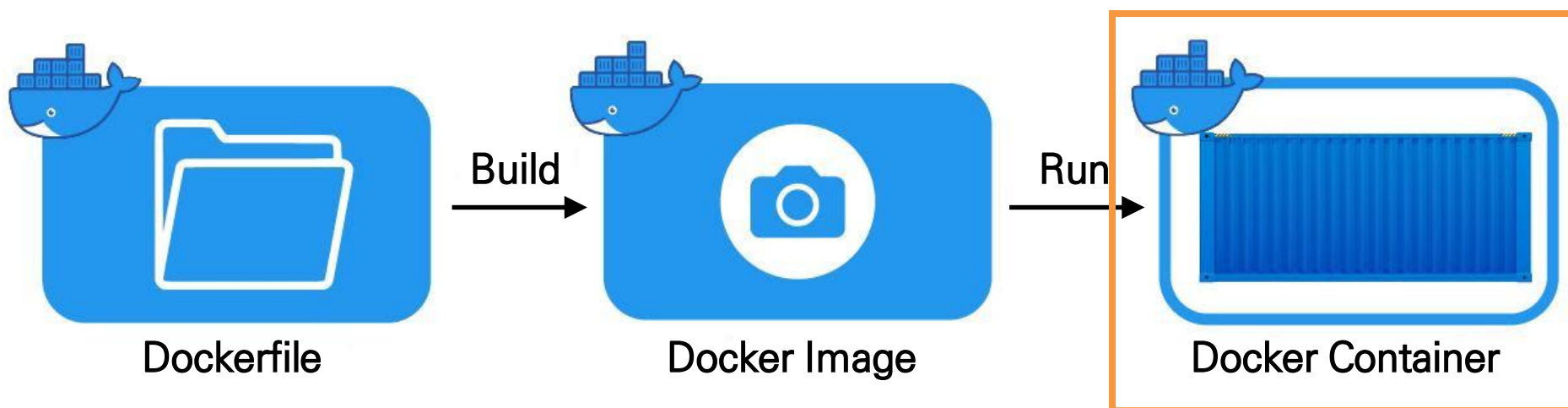
Docker image contains files and settings required to run the container and is created by building docker files

04 Docker Container Definition

Docker Container

Source: JFrog

Docker container



✓ Docker Container

Docker containers run applications in isolated spaces and run docker containers by running docker images

Creating a Dockerfile

Docker File Example (1/3)

```
FROM tomcat:9-jre8-alpine
WORKDIR /usr/local/tomcat
COPY server.xml ./conf
RUN rm -rf ./webapps/*
ARG JAR_FILE=*.war
COPY ${JAR_FILE} ./webapps/edu-msa-board-1.0.0.war

EXPOSE 28082
```

- ✓ Base image on which the FROM container is based
- ✓ Setting the working directory to run the WORKDIR command
- ✓ Copying files to containers when building COPY containers

Creating a Dockerfile

Docker File Example (2/3)

```
FROM tomcat:9-jre8-alpine
WORKDIR /usr/local/tomcat
COPY server.xml ./conf
RUN rm -rf ./webapps/*
ARG JAR_FILE=*.war
COPY ${JAR_FILE} ./webapps/edu-msa-board-1.0.0.war
EXPOSE 28082
```

- ✓ Commands to run on the RUN base image
- ✓ Set variable values for ARG files
- ✓ Port number to expose outside EXPOSE

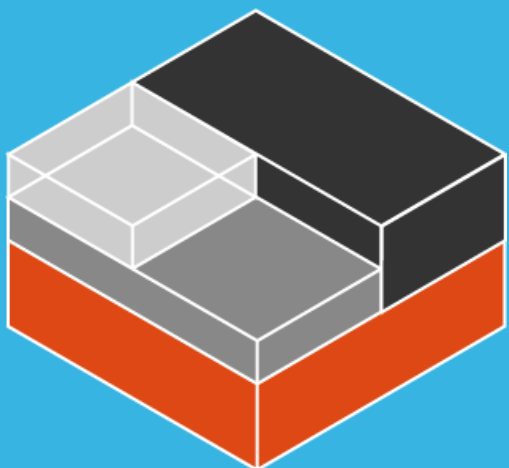
Creating a Dockerfile

Docker File Example (3/3)

```
FROM openjdk:8-jdk-alpine
ARG JAR_FILE=edu-msa-zuul-1.0.0.war
COPY ${JAR_FILE} edu-msa-zuul-1.0.0.war
COPY application.yml /application.yml
ENTRYPOINT ["java","-jar","-Dspring.config.location=application.yml","/edu-msa-zuul-1.0.0.war"]
```

- ✓ ENTRYPOINT
Specify the command to run when the container runs (change profile values)

LXC VS Docker



LXC

- ✓ Container runs multiple processes

VS



Docker

- ✓ Container runs a single process

LXC vs Docker

LXC

- ✓ One container runs multiple processes
- ✓ Run different applications in one container
- ✓ You can run multiple processes by adding init processes that manage multiple processes

Docker

- ✓ One container runs a single process
- ✓ Run only one application in one container
- ✓ Lightweight and fast to run because it runs only one application
- ✓ Less burden on the system
- ✓ Excellent portability of containers

A top-down view of a wooden desk with a laptop, glasses, a coffee cup, and a small plant. The laptop is on the right side, showing the keyboard and trackpad. A pair of black-rimmed glasses is in the center. A white coffee cup with a yellow handle is at the bottom right. A small green succulent is at the top center. The text '03. K8s Technology Introduction' is overlaid on the left side of the image.

03. K8s Technology Introduction

Kubernetes



What is Kubernetes?



kubernetes

Kubernetes

Container orchestration system that efficiently deploys containers and continuously checks and manages container status

Kubernetes

Kubernetes Features

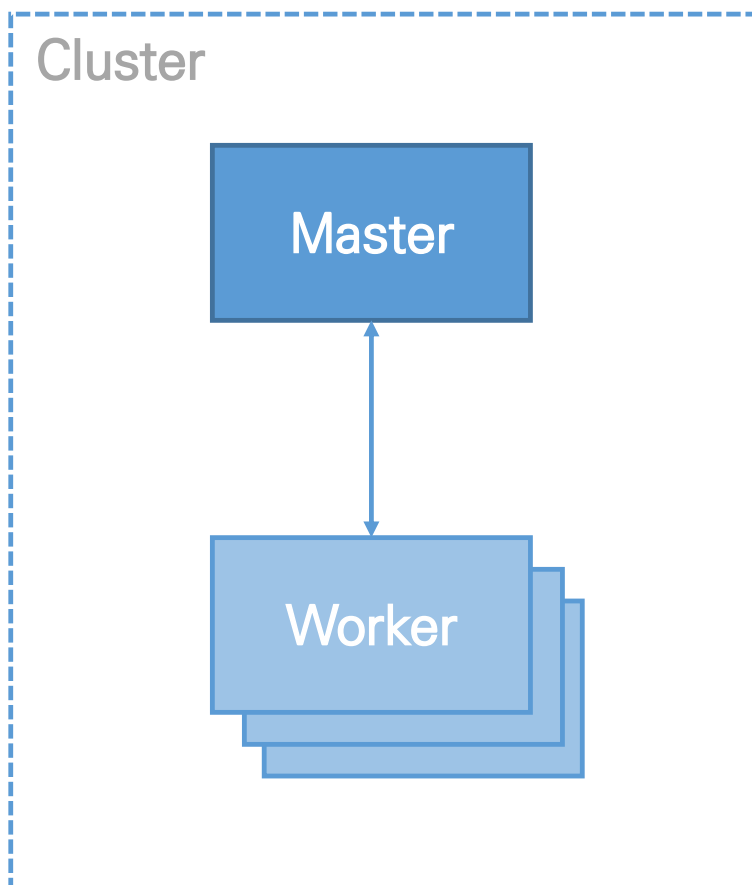


kubernetes

- ✓ Rapid Deployment
- ✓ Declarative API
- ✓ Flexibility and Scalability
- ✓ High Availability and Performance Management
- ✓ Run anywhere
- ✓ Available in a variety of environments
- ✓ Simplify application deployment
- ✓ Continuous health check and self-healing
- ✓ Auto Scaling

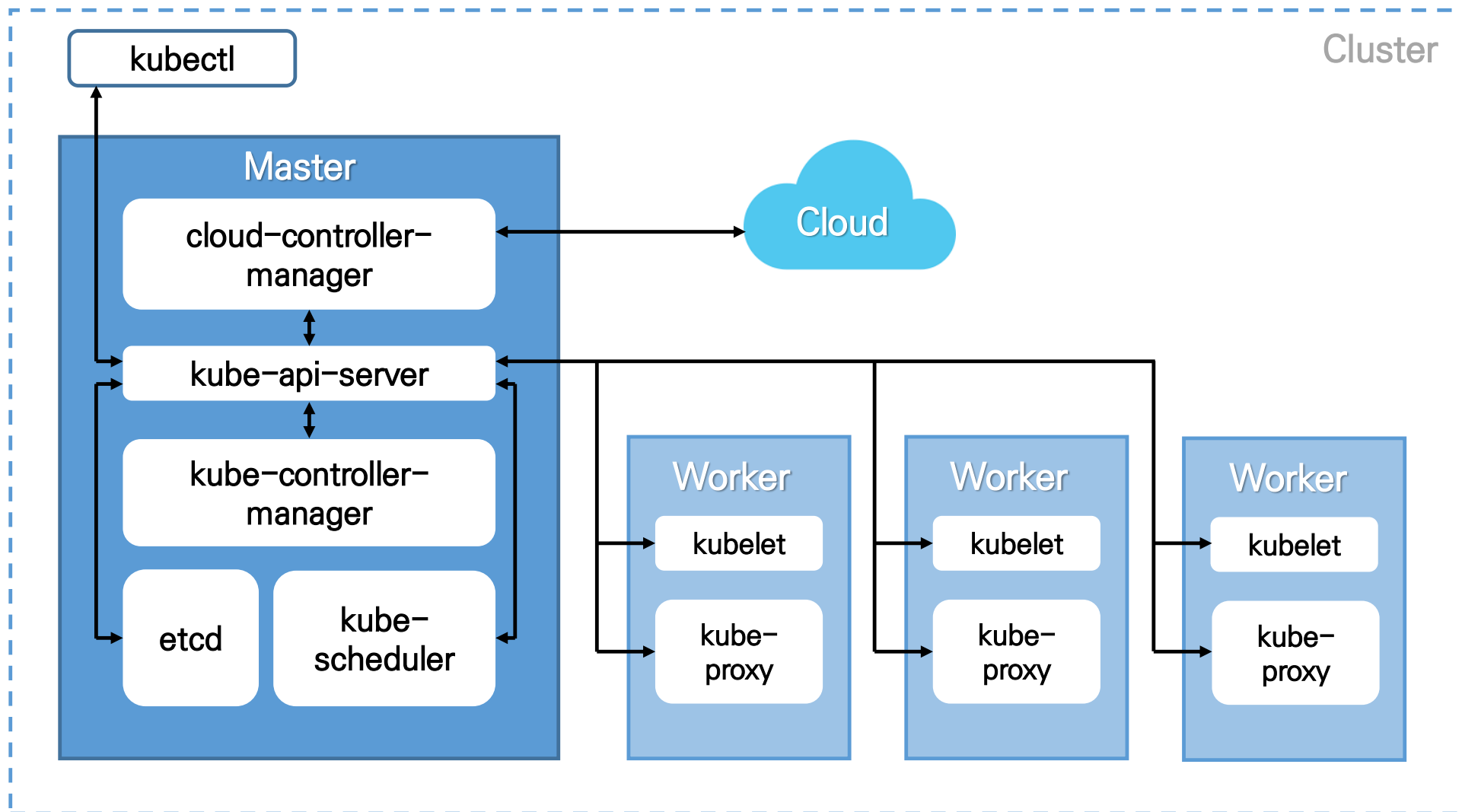
Kubernetes Cluster Configuration

Cluster Configuration

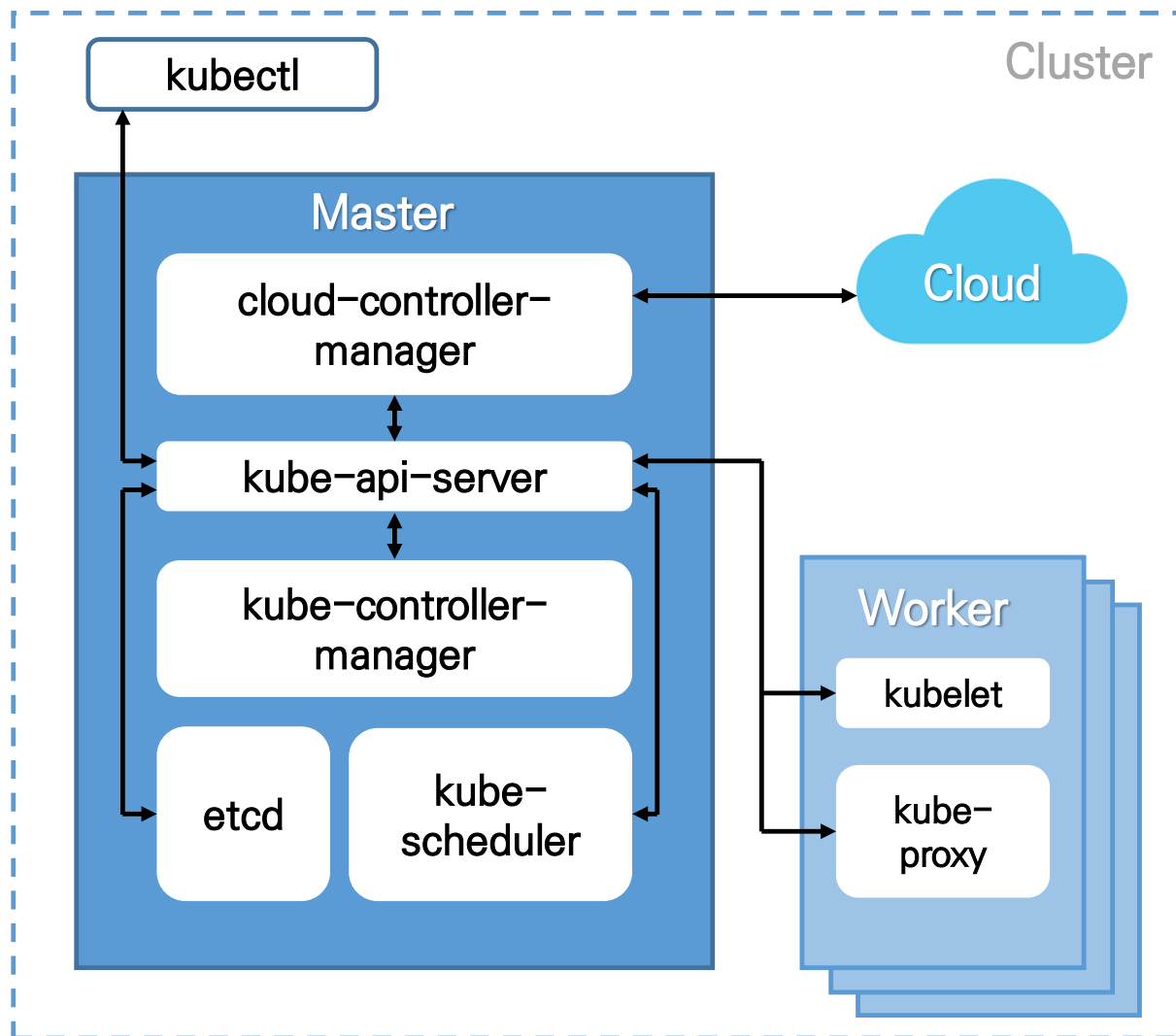


- ✓ **Cluster**
A cluster is a set of node machines to run containerized applications
- ✓ **Master**
A master node is a virtual machine that controls the entire cluster and manages all nodes inside
- ✓ **Worker**
Virtual machines that are commanded by the master, and that physical containers are created and work

Kubernetes Cluster Configuration

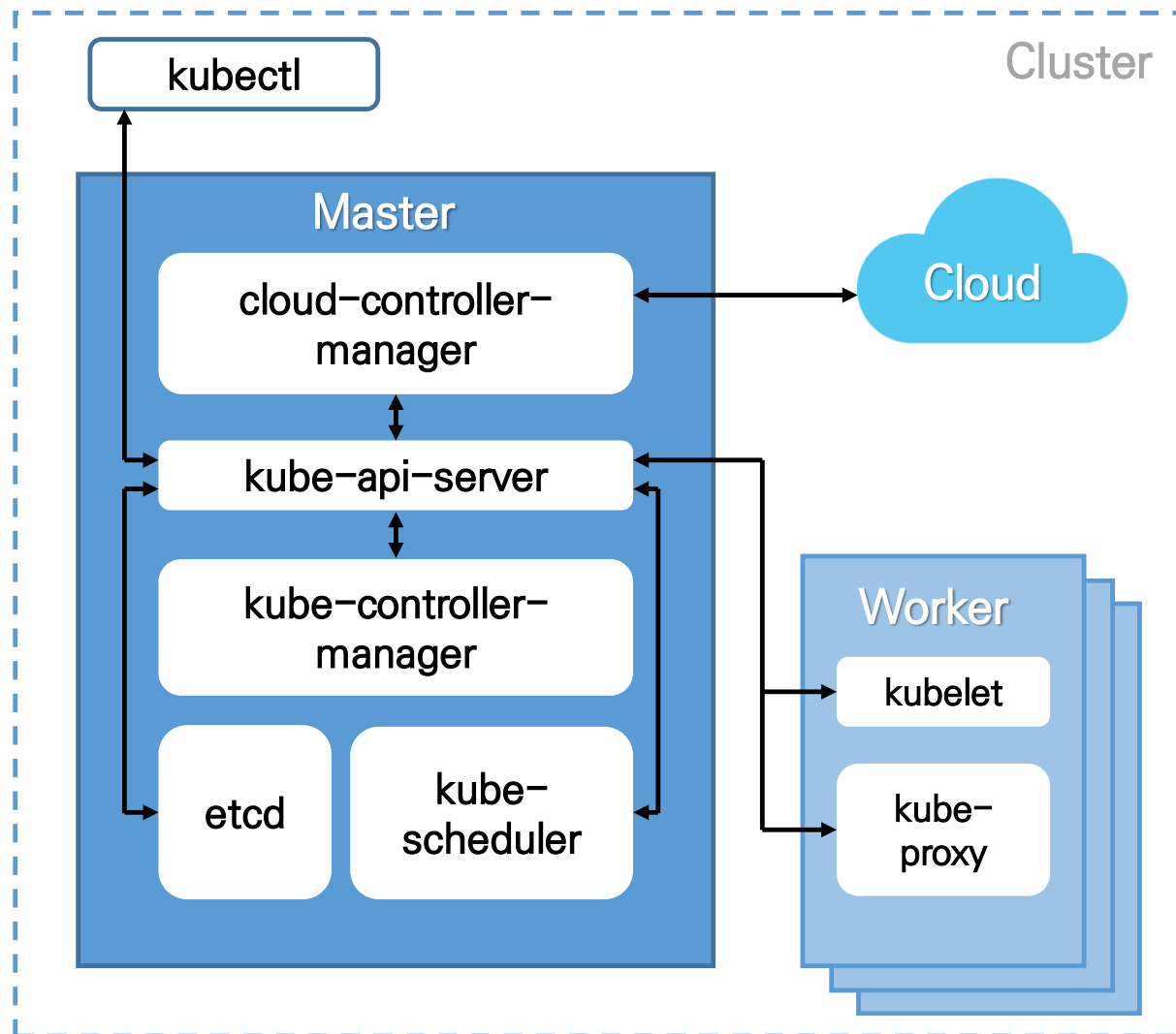


Kubernetes Cluster Configuration



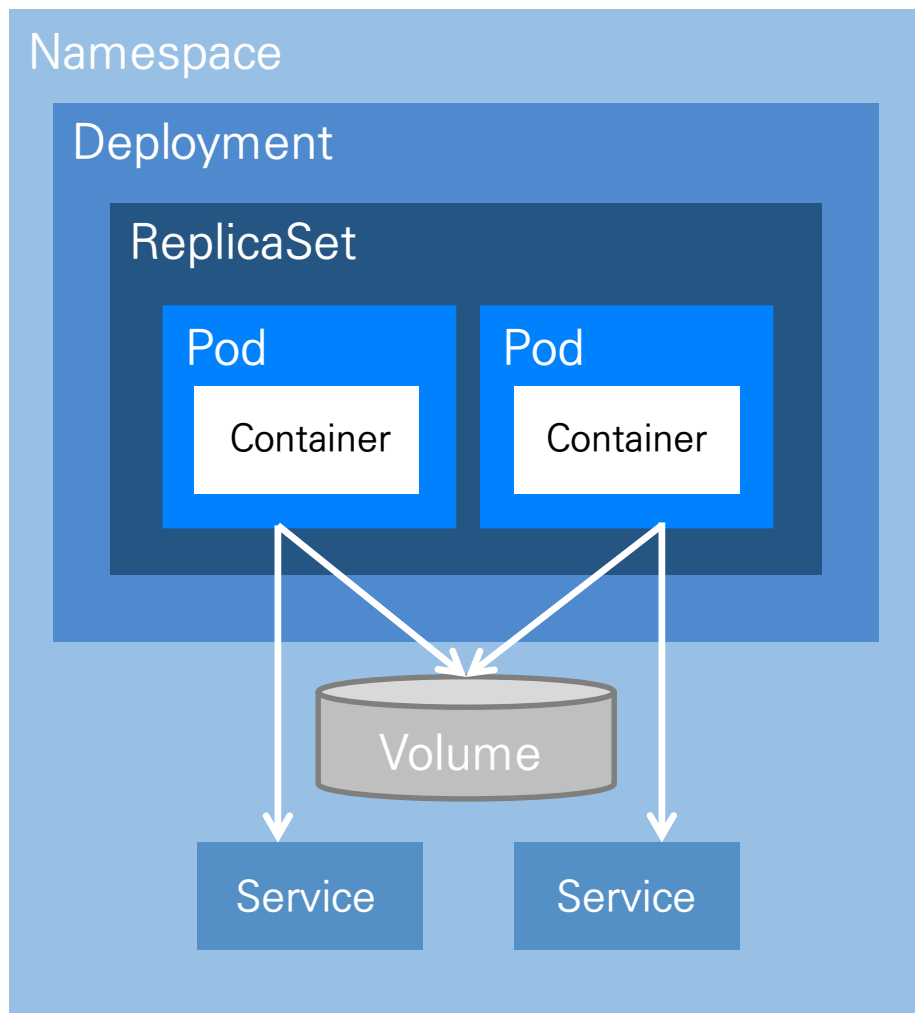
- ✓ **kube-api-server**
All Kubernetes communicate via kube-api-server
- ✓ **kube-controller-manager**
The role of controlling each controller that manages the pods
- ✓ **kube-scheduler**
Role of allocating resources to resource allocatable nodes
- ✓ **etcd**
Key-value storage that stores all the detailed data in the cluster

Kubernetes Cluster Configuration



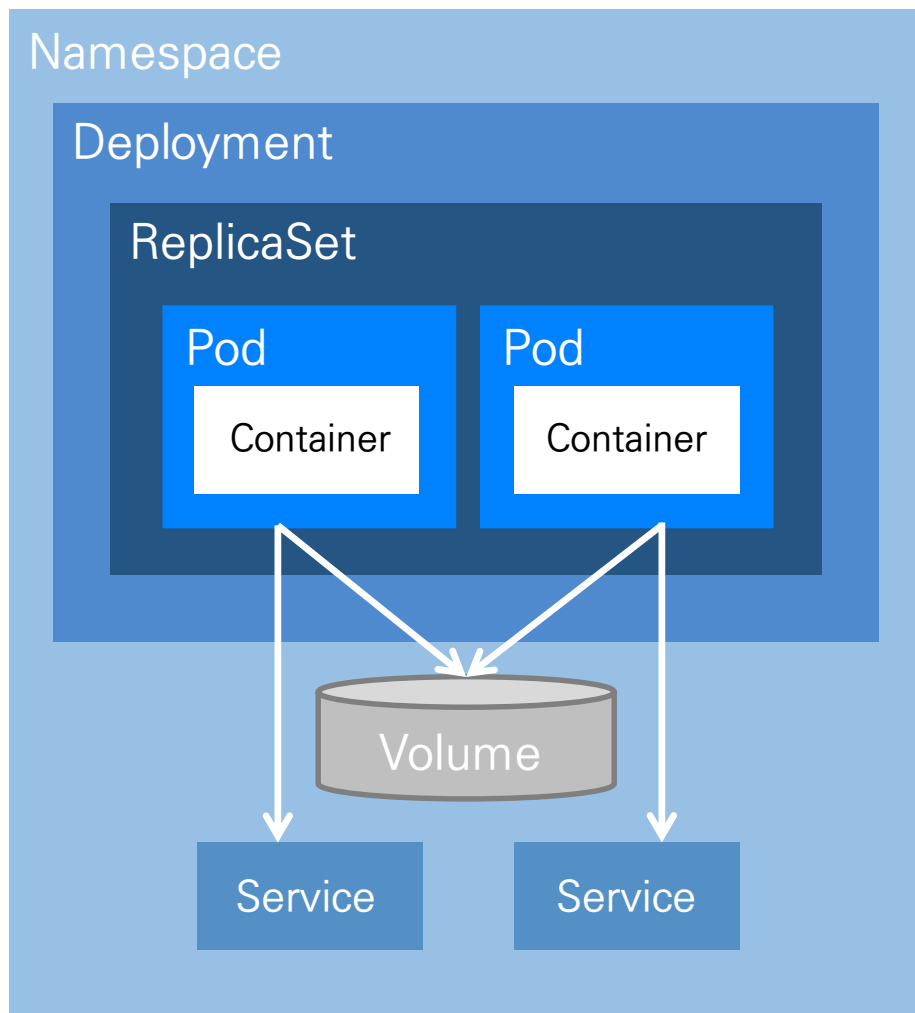
- ✓ **cloud-controller-manager**
Associate and manage controllers with cloud services
- ✓ **kubelet**
Agents running on all nodes and communicating with the master's kube-api-server through container execution and continuous health checks
- ✓ **kube-proxy**
Set up and manage virtual networks inside the cluster

Introducing Resources



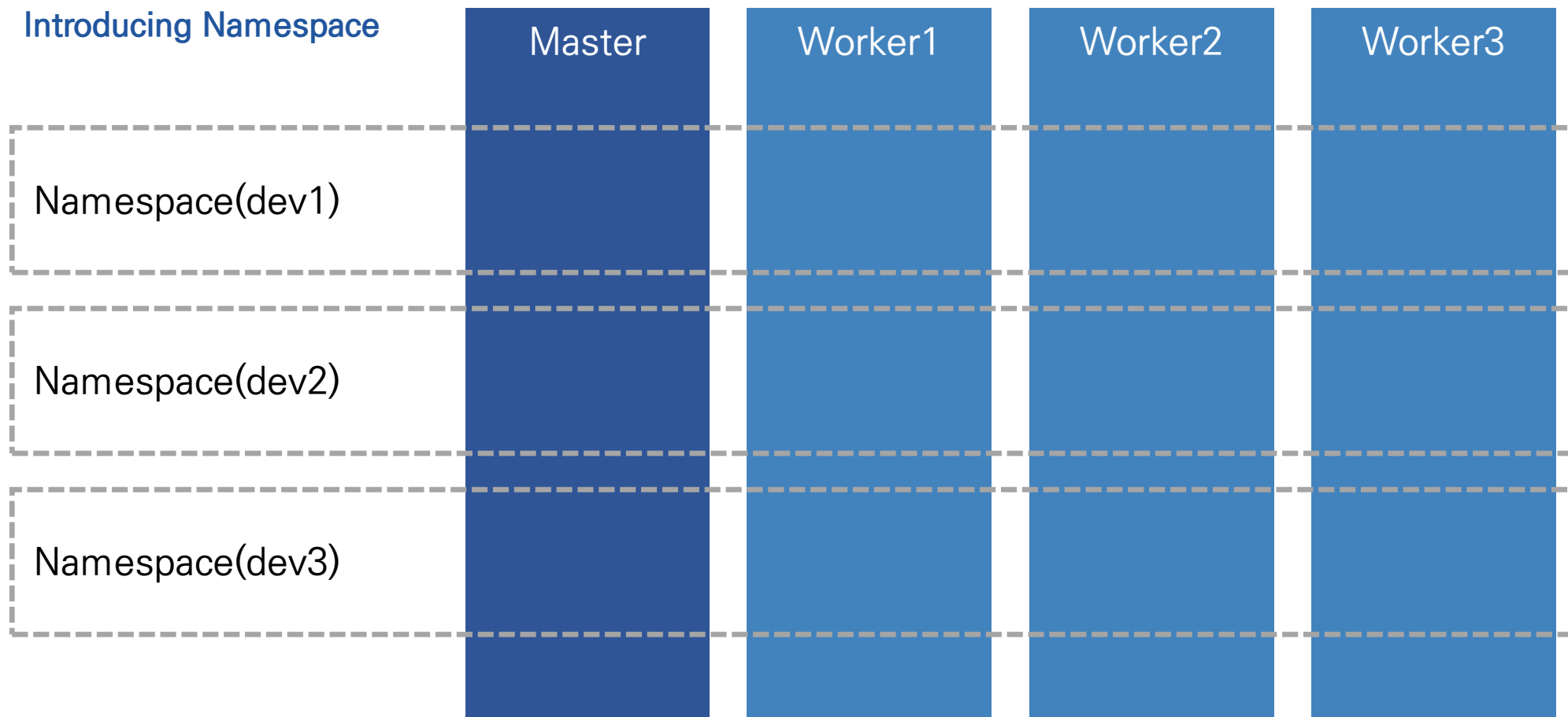
- ✓ **Namespace**
Resources that divide one identical physical cluster into multiple logical units
- ✓ **Deployment**
Resources that manage the replica set while distributing and managing the pods that need to be run
- ✓ **ReplicaSet**
Resources to maintain and manage the number of pods
- ✓ **Pod**
In fact, the container runs on the pod and the minimum unit to turn the container around

Introducing Resources



- ✓ **Service**
Resources that provide IP for access to internal pods outside the cluster
- ✓ **Volume**
As part of the pod, containers within the same pod can share volumes and maintain the data of the container

Introducing Namespace



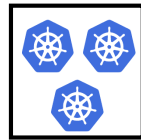
- ✓ Namespaces are resources separated by logical units of physical master/worker nodes (e.g. only authorized namespaces are accessible to each user) Only dev1 can access the privileged namespaces

Introducing Resources

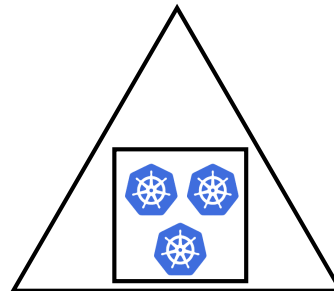
Introducing Resources



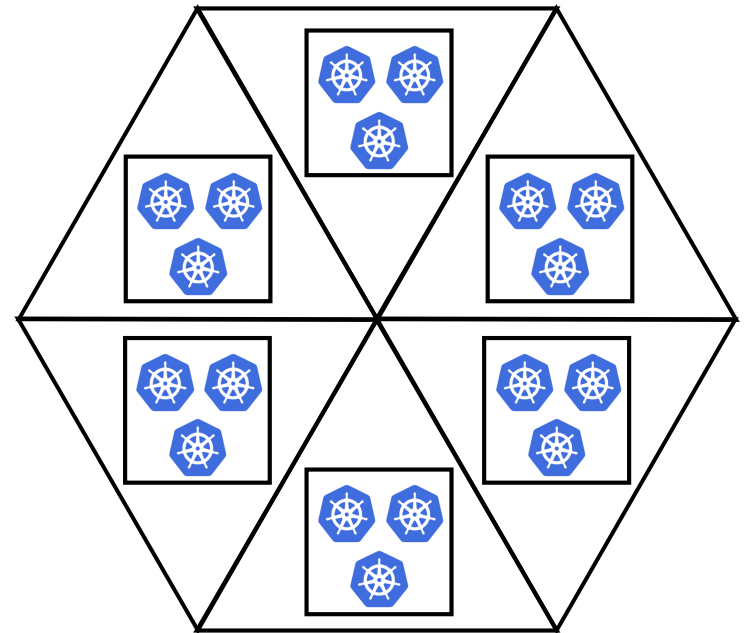
Container



Pod



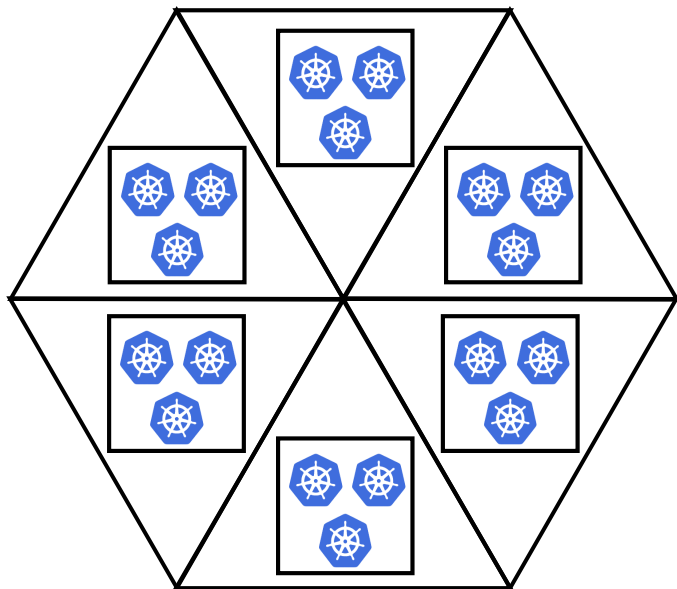
ReplicaSet



Deployment

Introducing Deployment

Deployment Features

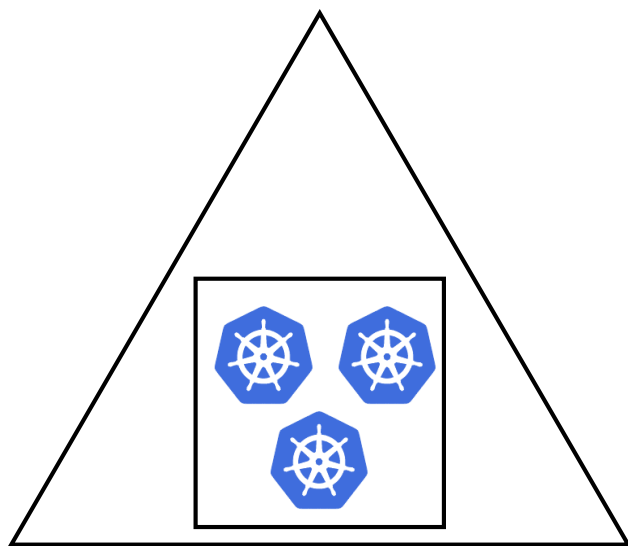


Deployment

- ✓ The most basic controller used when deploying apps with no status
- ✓ Manage Replica Sets
- ✓ Fine-grained management of app deployment
- ✓ Maintaining the number
- ✓ Rolling update on deployment

Introducing ReplicaSet

ReplicaSet Features

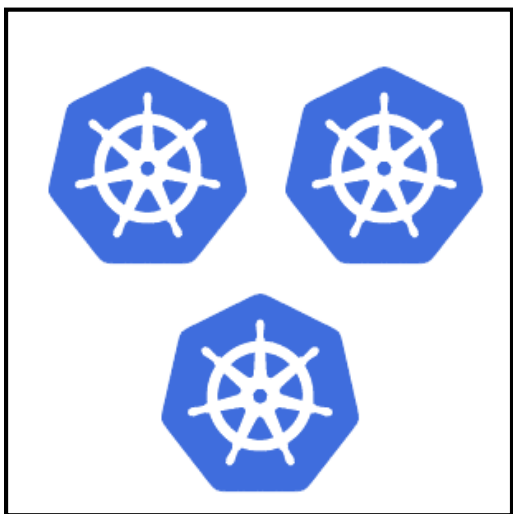


ReplicaSet

- ✓ Maintain and manage the number of pods
- ✓ Keep running pods stable
- ✓ Ensuring Availability for Pods Count

Introducing Pod

Pod Features

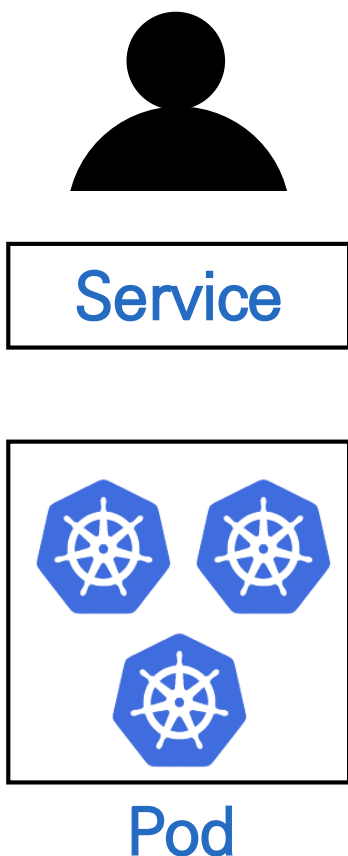


Pod

- ✓ Where the container runs directly
- ✓ Minimum unit for turning the container
- ✓ One or more container groups
- ✓ Manage containers on a per-pod basis rather than directly managing them

Introduction to Service

Service Features



- ✓ Accessing internal pods from outside the cluster
- ✓ Fixed IP Address Assignment
- ✓ Support for load balancing between pods
- ✓ Supports one or more ports

Persistent Volume 소개

Persistent Volume Features



Persistent Volume

- ✓ Independent volumes with separate lifecycles regardless of specific pods
- ✓ Provisioning is created in two ways: static and dynamic
- ✓ Assign PVs to meet requests when requested from PVCs
- ✓ PV to PVC mapping binds to 1:1 relationship

Introducing Persistent Volume Claim

Persistent Volume Claim Features



Persistent Volume Claim

- ✓ Requests made by users to PV
- ✓ Pads to PV
- ✓ Set storage capacity, access method, collector, etc. on demand
- ✓ PV to PVC mapping binds to 1:1 relationship

Introducing Storage Class

Storage Class Features

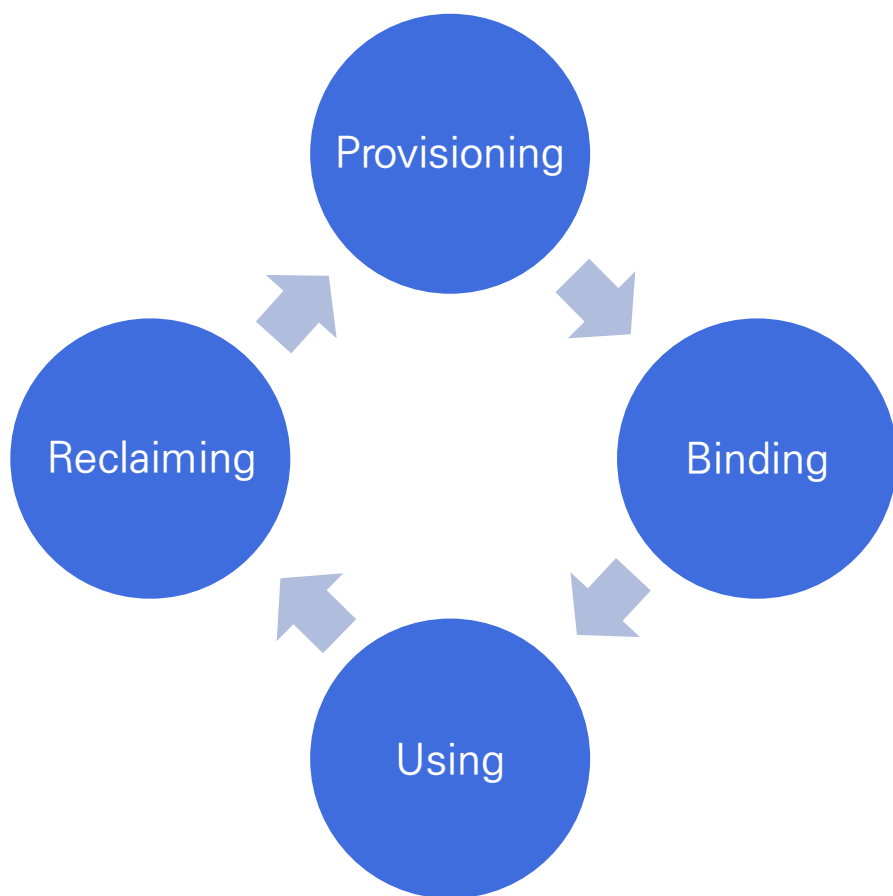


StorageClass

- ✓ Resources that define what kind of storage PV has
- ✓ Used when dynamic provisioning of PV, at least one storage class must be pre-created to enable dynamic provisioning
- ✓ Predefined storage classes allow you to create appropriate PVs immediately

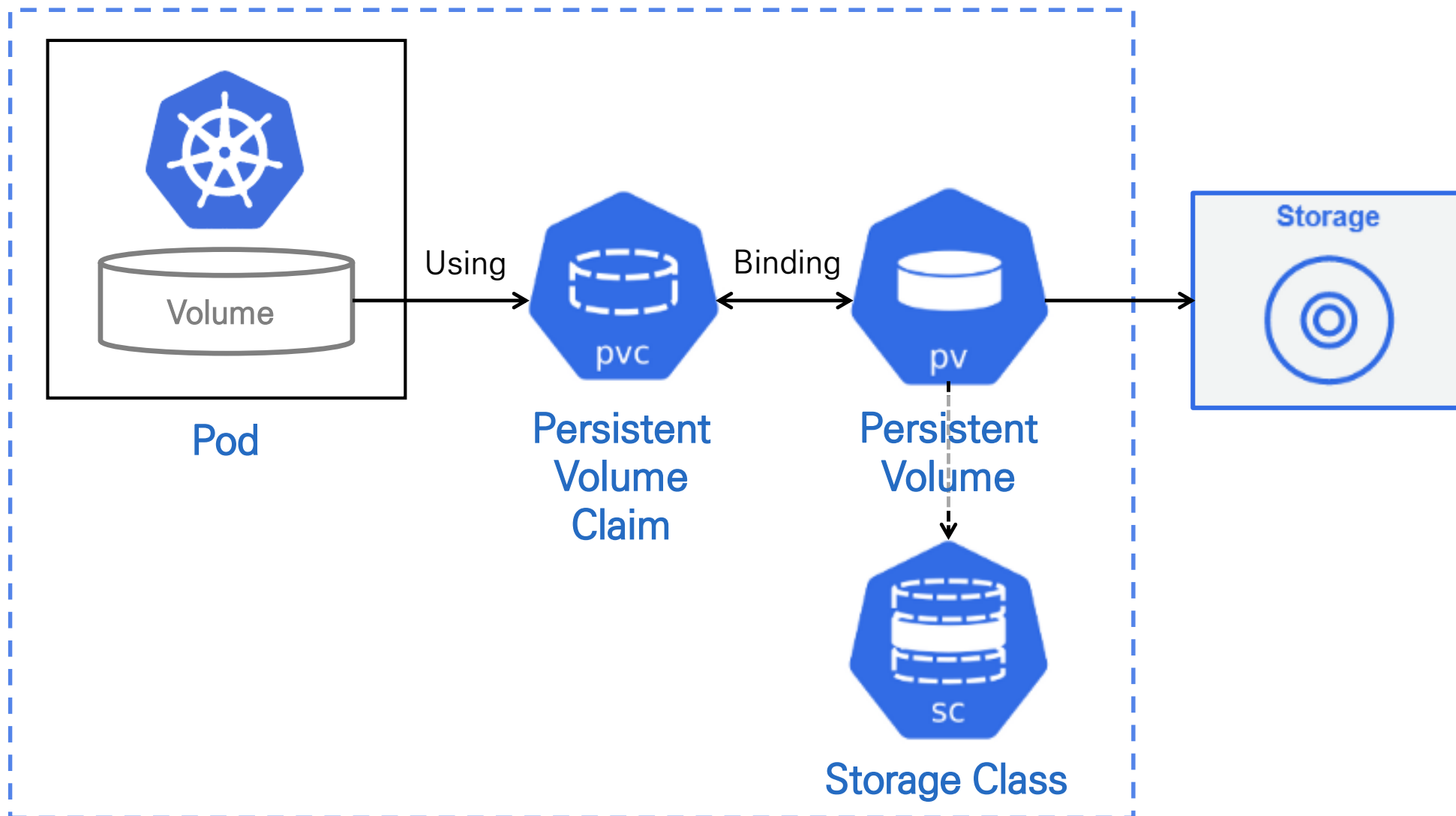
PV, PVC Lifecycle Introduction

the life cycle



- ✓ **Provisioning**
There are two methods, static and dynamic, and the steps to generate PVs
- ✓ **Binding**
When the PVC requests the desired approach, storage capacity, etc., the appropriate PV is connected
- ✓ **Using**
Steps where the PVCs set in the pod continue to be used while the pod is maintained
- ✓ **Reclaiming**
Process in which used PVCs are deleted and used PVs are initialized

Volume Structure



Introducing ConfigMap

Config Map Features



ConfigMap

- ✓ Provide configuration separately from containers
- ✓ A kind of storage role that stores key-value type of setup information
- ✓ Can have values from simple characters to large settings files
- ✓ Multiple config maps can be used for the same pod

Introducing Secret

Secret characteristics

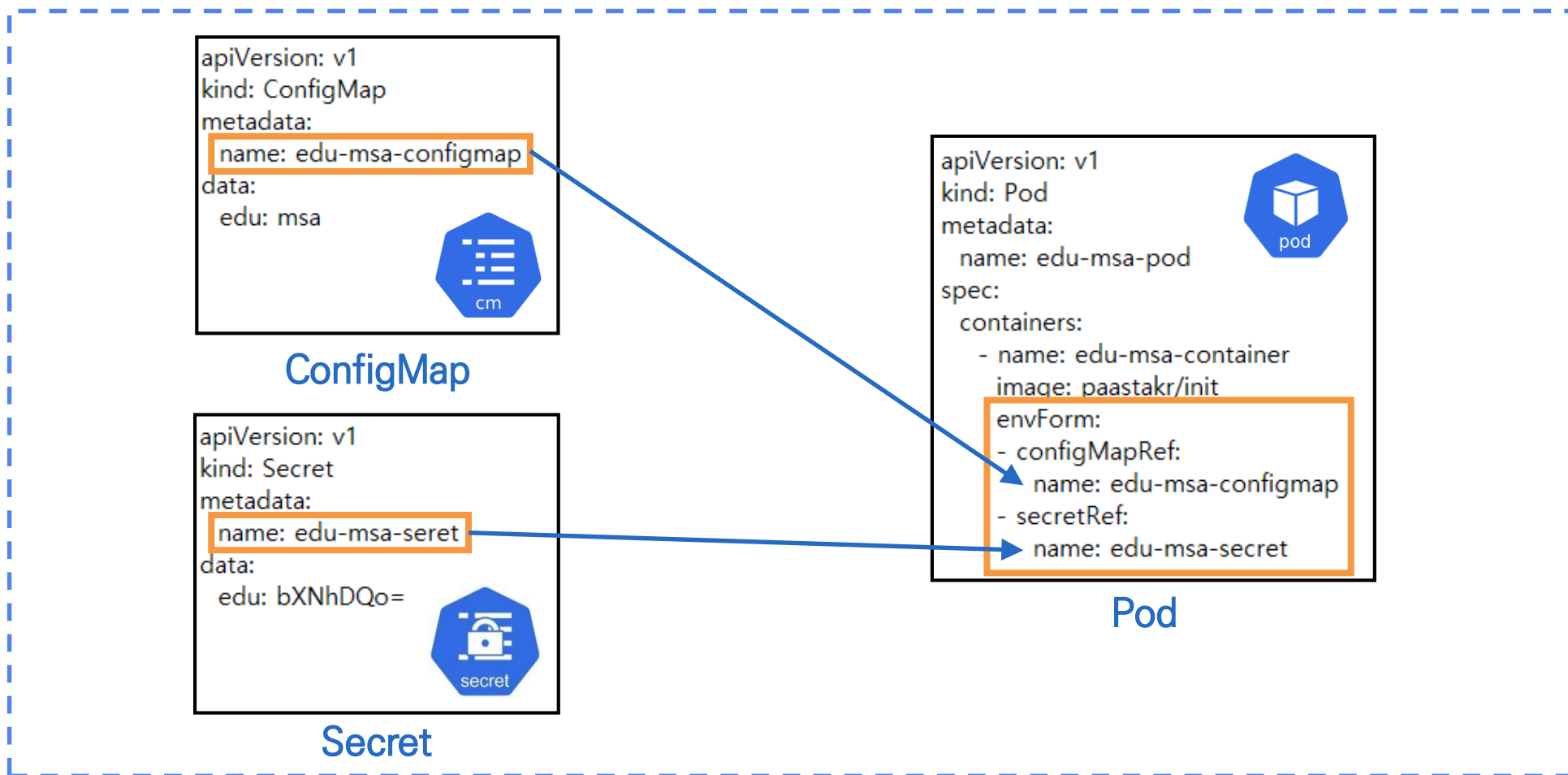


Secret

- ✓ The role of storing and managing information such as credentials and private encryption keys that require security
- ✓ Create with Base64 Encoding
- ✓ A kind of storage role that stores key-value type of setup information
- ✓ Stored in memory
- ✓ Maximum size of individual secret is defined up to 1MB
- ✓ All Pods have auto-connected secret volumes

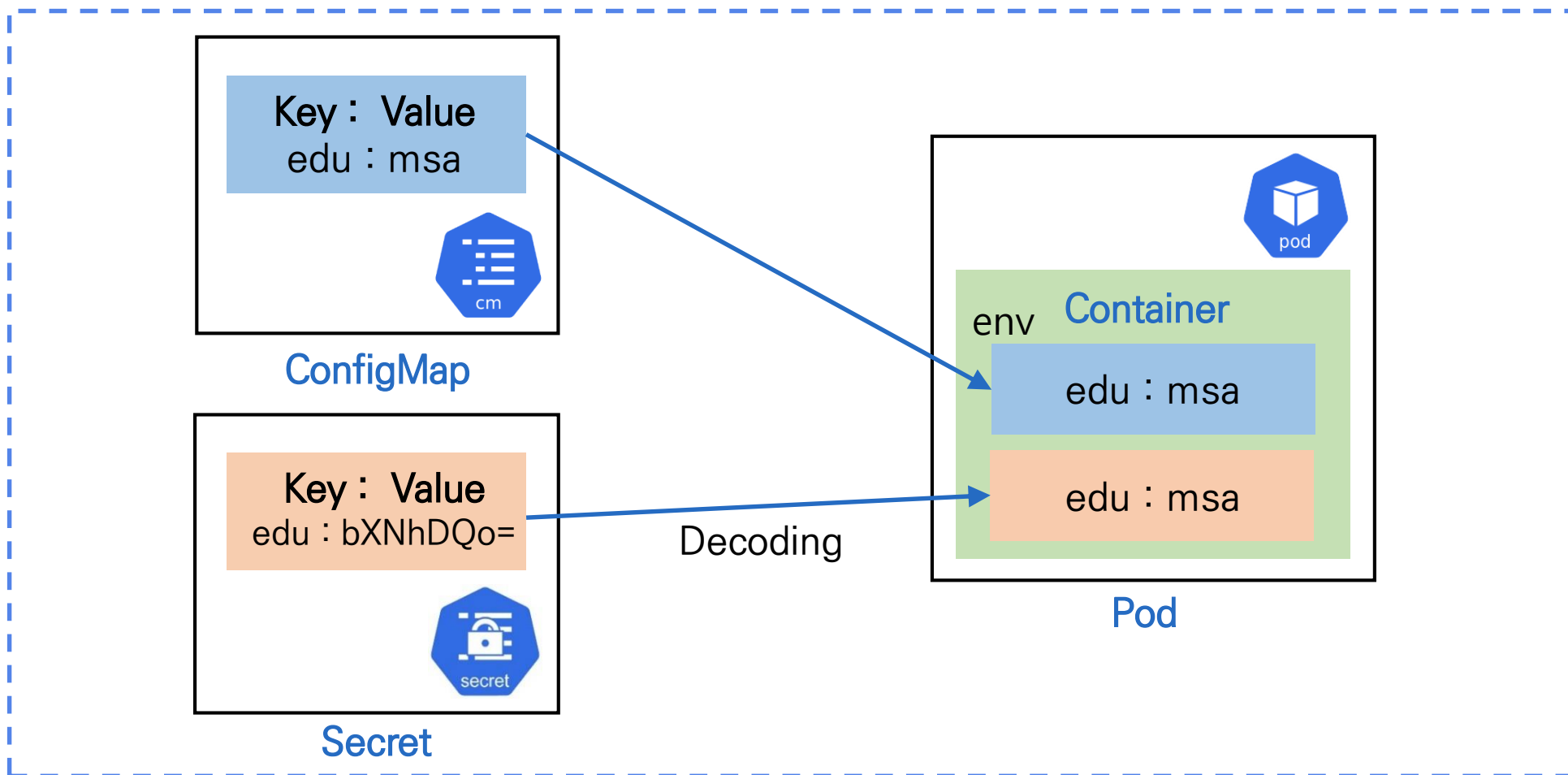
ConfigMap, introducing how to use Secret

- ✓ Enter config map and secret information when creating a pod



ConfigMap, introducing how to use Secret

- ✓ The container environment variable in the pod contains data from the config map and data from the decoded secret



Managing Kubernetes Resources

Kubernetes Resource Management

- ✓ Kubernetes resources are managed using the kubectl command line tool with configuration files written in YAML or JSON

```
apiVersion: v1
kind: Pod
metadata:
  name: paasta-pod
  labels:
    app: paasta-label
spec:
  containers:
  - name: paasta-container
    image: paasta/msa
```

YAML

```
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "name": "paasta-pod",
    "labels": {
      "app": "paasta-label"
    }
  },
  "spec": {
    "containers": [
      {
        "name": "paasta-container",
        "image": "paasta/msa"
      }
    ]
  }
}
```

JSON

```
|:~>kubectl create -f paasta.yaml
pod/paasta-pod created

|:~>kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
paasta-pod    1/1     Running   0           7s

|:~>kubectl delete -f paasta.yaml
pod "paasta-pod" deleted
```

kubectl command

To create a yaml file

Example of Deployment (1/7)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- ✓ **apiVersion**
Specify the version of the Kubernetes API to use to create specific resources
- ✓ **kind**
Specify the type of resource that you want to create and what kind of resource you want to create

To create a yaml file

Example of Deployment (2/7)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- ✓ [metadata](#)
Data that gives a resource a name and uniquely separates it
- ✓ [metadata.name](#)
Specify the name of the resource
- ✓ [metadata.labels](#)
Used to list or search only certain Kubernetes resources that are configured as key-value pairs
- ✓ [metadata.labels.app](#)
Set a label for a resource

To create a yaml file

Example of Deployment (3/7)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- ✓ `spec`
Specifically define the content of the resource you want to create
- ✓ `spec.replicas`
Set the number of pods you want to float
- ✓ `spec.selector`
Define which pods should be controlled
- ✓ `spec.selector.matchLabels`
Map to the same settings as `metadata.labels` to count the containers in the pod with the same label to measure the current state

To create a yaml file

Example of Deployment (4/7)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
```

- ✓ `spec.template`
Set information about which pod to run
- ✓ `spec.template.metadata`
Run a pod with the same label by mapping it to the same settings as `metadata.labels`

To create a yaml file

Example of Deployment (5/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
  - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- ✓ `spec.template.spec`
Settings for the container you want to
- ✓ `spec.template.spec.containers[]`
Set the name, image, port number, etc. of the container to run
- ✓ `spec.template.spec.containers[].name`
Specify a container name

Introducing how to create Kubernetes yaml

To create a yaml file

Example of Deployment (6/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
  - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- ✓ `spec.template.spec.containers[].image`
Specify the image to be deployed as a container
- ✓ `spec.template.spec.containers[].imagePullPolicy`
Specify an image download policy ("Always" downloads regardless of image)
- ✓ `spec.template.spec.containers[].ports[]`
Specify container port number

Introducing how to create Kubernetes yaml

To create a yaml file

Example of Deployment (7/7)

```
spec:
  containers:
  - name: board-msa
    image: paastakr/edu-msa-board:latest
    imagePullPolicy: Always
    ports:
    - containerPort: 28082
  imagePullSecrets:
  - name: edu-msa-secret
  nodeSelector:
    kubernetes.io/hostname: paas-ta-worker-1
```

- ✓ `spec.template.spec.imagePullSecret`
Credentials for accessing the image store
- ✓ `spec.template.spec.nodeSelector`
Specify the specific node on which to deploy the pod

To create a yaml file

Service example (1 of 2)

```
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
    - nodePort: ${EDU_MSA_BOARD}
      port: 28082
      protocol: TCP
      targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

- ✓ `spec.ports[].nodePort`
Set an externally accessible port number (external port)
- ✓ `spec.ports[].port`
Set container port number (internal port)
- ✓ `spec.ports[].protocol`
Set protocol method
- ✓ `spec.ports[].targetport`
Set the container port number you want to access

To create a yaml file

Service Example (2/2)

```
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
    - nodePort: ${EDU_MSA_BOARD}
      port: 28082
      protocol: TCP
      targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

- ✓ **spec.type**
Set how you want to expose to the outside world ("NodePort" is a method that can be accessed externally through the port number)

Introducing how to create Kubernetes yaml

To create a yaml file

Consolidate yaml files

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  replicas: 1
  selector:
    matchLabels:
      app: board-msa
  template:
    metadata:
      labels:
        app: board-msa
    spec:
      containers:
        - name: board-msa
          image: paastakr/edu-msa-board:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 28082
---
apiVersion: v1
kind: Service
metadata:
  name: edu-msa-board
  labels:
    app: board-msa
spec:
  ports:
    - nodePort: 30201
      port: 28082
      protocol: TCP
      targetPort: 28082
  selector:
    app: board-msa
  type: NodePort
```

Deployment

Service

- ✓ To create multiple related yaml files as a single file, create them separated by a "---" in the middle

Using the kubectl Command

basic form

- ✓ kubectl command: https://kubernetes.io/ko/docs/reference/kubectl/_print/

Default Syntax

```
$ kubectl [COMMAND] [TYPE] {NAME} {FLAGS}
```

Explanation

- ✓ [COMMAND] The action you want to perform (create, apply, get, save, delete, edit, etc.)
- ✓ [TYPE] Resource type (all, pods, deployments, services, etc.)
- ✓ Resource name {NAME}
- ✓ {FLAGS} Optional flagging (-f FILENAME, -n NAMESPACE -wide, etc.)

Using the kubectl Command

Create (1/2)

Default Syntax

```
$ kubectl [COMMAND] {FLAGS} [FILE_NAME]
```

Explanation

Create resources for the yaml file (run where the yaml file is located during creation)

Example of use

```
$ kubectl apply -f nginx-deployment.yaml
```

```
I:~/kubeyaml>kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
```

Using the kubectl Command

Create (2/2)

Default Syntax

```
$ kubectl [COMMAND] {FLAGS} [FILE_NAME]
```

Explanation

Create resources for the yaml file (run where the yaml file is located during creation)

Example of use

```
$ kubectl create -f nginx-deployment.yaml
```

```
I:~#kubeyaml>kubectl create -f nginx-deployment.yaml  
deployment.apps/nginx created
```

Using the kubectl Command

Inquiry (1/4)

Default Syntax

\$ kubectl [COMMAND] [TYPE]

Explanation

Output default information for specific resources

Example of use

\$ kubectl get pods

```
I:~#kubeyaml>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-559d75d76b-4zhsz 1/1     Running   0           2m2s
nginx-559d75d76b-7b8s2 1/1     Running   0           2m2s
```

Using the kubectl Command

Inquiry (2/4)

Default Syntax

```
$ kubectl [COMMAND] [TYPE] {FLAGS}
```

Explanation

Output additional information for specific resources

Example of use

```
$ kubectl get pods -o wide
```

```
l:\#\kubeyaml>kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
nginx-559d75d76b-4zhsz             1/1     Running   0           2m17s  10.233.117.192  playpark-caas-worker-6             <none>            <none>
nginx-559d75d76b-7b8s2             1/1     Running   0           2m17s  10.233.117.191  playpark-caas-worker-6             <none>            <none>
```


Using the kubectl Command

Inquiry (3/4)

Default Syntax	\$ kubectl [COMMAND] [TYPE] {NAME}
----------------	------------------------------------

Explanation	Output default information for resources with a specific name
-------------	---

Example of use	\$ kubectl get pod nginx-559d75d76b-4zhsz
----------------	---

```
I:~#kubeyaml>kubectl get pod nginx-559d75d76b-4zhsz
NAME                READY   STATUS    RESTARTS   AGE
nginx-559d75d76b-4zhsz  1/1     Running   0           5m14s
```

Using the kubectl Command

Inquiry (4/4)

Default Syntax `$ kubectl [COMMAND] [TYPE] {NAME} {FLAGS}`

Explanation Output additional information for resources with a specific name

Example of use `$ kubectl get pod nginx-559d75d76b-4zhsz -o wide`

```
I:\kubeyaml>kubectl get pod nginx-559d75d76b-4zhsz -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE                   NOMINATED NODE   READINESS GATES
nginx-559d75d76b-4zhsz             1/1    Running   0          5m20s  10.233.117.192  playpark-caas-worker-6  <none>           <none>
```

Using the kubectl Command

Detailed Inquiry

Default Syntax

\$ kubectl [COMMAND] [TYPE] {NAME}

Explanation

Output detailed information for resources with a specific name

Example of use

\$ kubectl describe pod nginx-559d75d76b-4zhsz

```
l:~kubeyaml>kubectl describe pod nginx-559d75d76b-4zhsz
Name:          nginx-559d75d76b-4zhsz
Namespace:     paas-83cc724a-4425-4f1e-aab4-63a4f23fcd99-caas
Priority:       0
Node:          playpark-caas-worker-6/10.0.50.254
Start Time:    Fri, 29 Oct 2021 14:47:49 +0900
Labels:        app=nginx
               pod-template-hash=559d75d76b
Annotations:   cni.projectcalico.org/podIP: 10.233.117.192/32
               cni.projectcalico.org/podIPs: 10.233.117.192/32
               kubernetes.io/limit-ranger: LimitRanger plugin set: cpu, memory request for container nginx; cpu, memory limit for container nginx
Status:        Running
IP:            10.233.117.192
IPs:           IP: 10.233.117.192
Controlled By: ReplicaSet/nginx-559d75d76b
Containers:
```

Log Lookup

```
$ kubectl [COMMAND] {FLAGS} {POD_NAME}
```

Output log information for a resource with a specific name

```
$ kubectl logs -f edu-msa-zuul-647b787d7-g5st7
```

[illegible]

```
2021-10-29 09:15:55.805 INFO 1 --- [main] paasta.msa.zuul.ZuulApplication : No active profile set,
falling back to default profiles: default
2021-10-29 09:17:06.104 WARN 1 --- [main] o.s.boot.actuate.endpoint.EndpointId : Endpoint ID 'hystrix.st
ream' contains invalid characters, please migrate to a valid format.
```

Using the kubectl Command

Modifying the Editor (1 of 2)

Default Syntax

```
$ kubectl [COMMAND] [TYPE] {NAME}
```

Explanation

Modify resource information with a specific name

Example of use

```
$ kubectl edit deployment nginx
```

- ✓ When you enter the above command, an editor opens with information about the resource you want to modify
- ✓ Save Editor after modification and reflect modifications when exiting

Using the kubectl Command

Modifying the Editor (2 of 2)

```
spec:
  progressDeadlineSeconds: 600
  replicas: 2
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: nginx
  strategy:
```

```
spec:
  progressDeadlineSeconds: 600
  replicas: 3
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: nginx
  strategy:
```

```
I:~kubeyaml>kubectl edit deployment nginx
deployment.apps/nginx edited
```

```
I:~kubeyaml>kubectl get replicaset
NAME                DESIRED   CURRENT   READY   AGE
nginx-559d75d76b    3         3         3       47m
```

Using the kubectl Command

Modify yaml file (1/2)

- ✓ Redistribute again by modifying the yaml file that was distributed at creation time

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.13.12
          ports:
            - containerPort: 80
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 4
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.13.12
          ports:
            - containerPort: 80
```

Using the kubectl Command

Modify yaml file (2/2)

Default Syntax `$ kubectl [COMMAND] {FLAGS} {NAME}`

Explanation Modify resource using yaml file (if modified, run where yaml file is located)

Example of use `$ kubectl apply -f nginx-deployment.yaml`

```
I:~kubeyaml>kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx configured
```

```
I:~kubeyaml>kubectl get replicaset
NAME                DESIRED    CURRENT    READY    AGE
nginx-559d75d76b     4          4          4        30m
```


Using the kubectl Command

Delete (1/2)

Default Syntax	\$ kubectl [COMMAND] [TYPE] {NAME}
Explanation	Delete a specific resource that you want to delete
Example of use	\$ kubectl delete pod nginx-559d75d76b-4zhsz

```
I:~kubeyaml>kubectl delete pod nginx-559d75d76b-4zhsz  
pod "nginx-559d75d76b-4zhsz" deleted
```

Using the kubectl Command

Delete (2/2)

Default Syntax `$ kubectl [COMMAND] {FLAGS} [FILE_NAME]`

Explanation Delete resource using yaml file (run where yaml file is located when deleted)

Example of use `$ kubectl delete -f nginx-deployment.yaml`

```
I:~/kubeyaml>kubectl delete -f nginx-deployment.yaml
deployment.apps "nginx" deleted
```

Using the kubectl Command

Container access

Default Syntax	<code>\$ kubectl [COMMAND] {FLAGS} {NAME} /bin/bash</code>
Explanation	Accessing a working container for a particular pod
Example of use	<code>\$ kubectl exec -it mysql-board-msa-5967cb9786-9vwgz /bin/bash</code>

```
I:~kubeyaml>kubectl exec -it mysql-board-msa-5967cb9786-9vwgz /bin/bash
kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a future version. Use kubectl exec
[POD] -- [COMMAND] instead.
root@mysql-board-msa-5967cb9786-9vwgz:/#
```

01 Installing the Docker CLI

Installing the Docker CLI

What is Docker Desktop?



- ✓ Docker designed to run on Windows OS
- ✓ Enable Windows native Hyper-V virtualization and networking
- ✓ The Fastest and Most Reliable Way to Develop Docker Applications in Windows
- ✓ Docker Desktop supports running both Linux and Windows docker containers

01 Installing the Docker CLI

Installing the Docker CLI

Docker Desktop Installation (1/6)

- ✓ Docker Desktop: <https://docs.docker.com/desktop/windows/install/>



The screenshot shows the Docker Docs website. The top navigation bar is blue with the Docker logo and 'docker docs' text. A search bar is next to it. The main navigation menu includes 'Home', 'Guides', 'Manuals', 'Reference', and 'Samples'. The breadcrumb trail is 'Home / Manuals / Docker Desktop / Windows / Install Docker Desktop for Windows'. The left sidebar shows a tree view with 'Docker Desktop' expanded, containing 'Overview', 'Mac', 'Windows', 'Install Docker Desktop for Windows' (highlighted), 'User manual', 'Networking', 'Logs and troubleshooting', and 'Docker Desktop WSL 2 backend'. The main content area has the title 'Install Docker Desktop on Windows' with an estimated reading time of 8 minutes. The text below the title says 'Welcome to Docker Desktop for Windows. This page contains information about Docker Desktop for Windows and instructions to install and update Docker Desktop for Windows.' A blue button labeled 'Download Docker Desktop for Windows' is visible. A grey callout box with a checkmark and the text 'Docker Desktop for Windows Click' is overlaid on the button.

Installing the Docker CLI

Docker Desktop Installation (2/6)

The screenshot shows the Windows Settings application, specifically the 'Programs and Features' section. The 'Windows Features' link is highlighted in the left sidebar. A 'Windows Features' dialog box is open, displaying a list of features. The 'Hyper-V' feature is checked, and its checkbox is highlighted with a red box. Other features like '.NET Framework 3.5', '.NET Framework 4.8 Advanced Services', and 'Active Directory LDS' are also visible. At the bottom of the dialog, there are '확인' (OK) and '취소' (Cancel) buttons.

Windows 기능

Windows 기능 켜기/끄기

기능을 사용하려면 해당 확인란을 선택하고 기능을 사용하지 않으려면 확인란의 선택을 취소하십시오. 확인란이 검은 사각형으로 채워진 경우는 해당 기능의 일부만이 사용되고 있는 것입니다.

- ☒ .NET Framework 3.5(.NET 2.0 및 3.0 포함)
- ☒ .NET Framework 4.8 Advanced Services
- ☐ Active Directory LDS(Lightweight Directory Services)
- ☒ **Hyper-V**
- ☒ Internet Explorer 11
- ☒ Linux용 Windows 하위 시스템
- ☐ Microsoft Defender Application Guard
- ☒ Microsoft PDF로 인쇄
- ☒ Microsoft XPS Document Writer
- ☐ MSMQ(Microsoft Message Queue) Server
- ☐ MultiPoint Connector

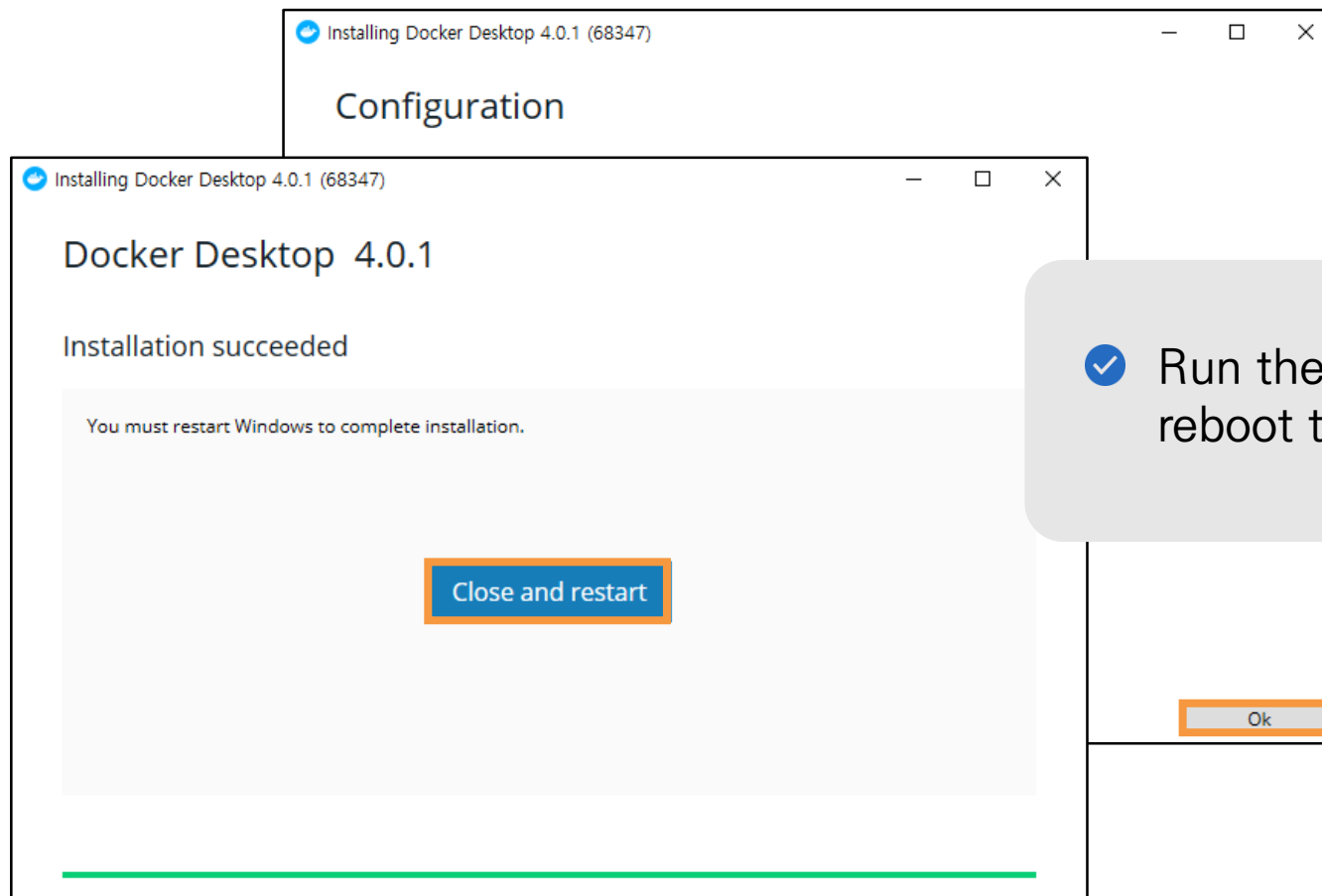
확인 취소

✓ Ensure Hyper-V is checked in Windows features before installing the Docker CLI

01 Installing the Docker CLI

Installing the Docker CLI

Docker Desktop Installation (3/6)

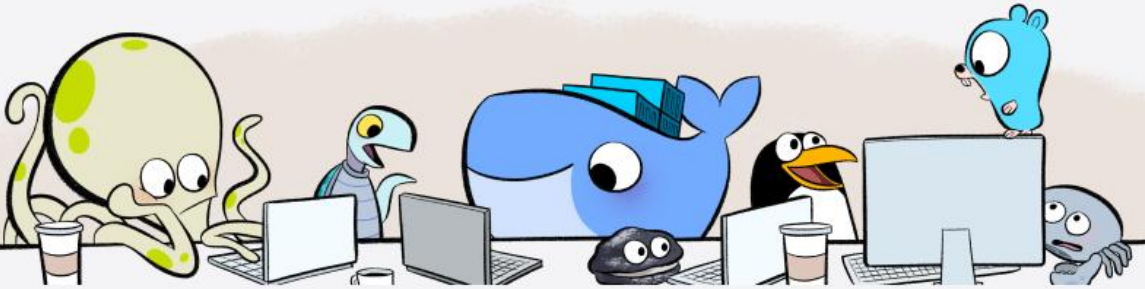


- ✓ Run the downloaded file to reboot the PC after installation

01 Installing the Docker CLI

Installing the Docker CLI

Docker Desktop Installation (4/6)



Our Service Agreement has Changed

We've updated the [Docker Subscription Service Agreement](#). Please read the [Blog](#) and [FAQs](#) to learn how companies using Docker Desktop may be affected. By checking "I accept the terms" you agree to the [Subscription Service Agreement](#), the [Data Processing Agreement](#), and the [Data Privacy Policy](#).

Here's a summary of key changes:

- Our Docker Subscription Service Agreement include a change to the terms of use for Docker Desktop
 - It **remains free** for small businesses (fewer than 250 employees AND less than \$10 million annual revenue), personal use, education, and non-commercial open source projects.
 - It requires a paid subscription for professional use in larger enterprises.
- The effective date of these terms is August 31, 2021. There is a **grace period** until January 31, 2022 for those that will require a paid subscription to use Docker Desktop.
- The existing Docker Free subscription has been renamed **Docker Personal** and we have introduced a

☒ I accept the terms

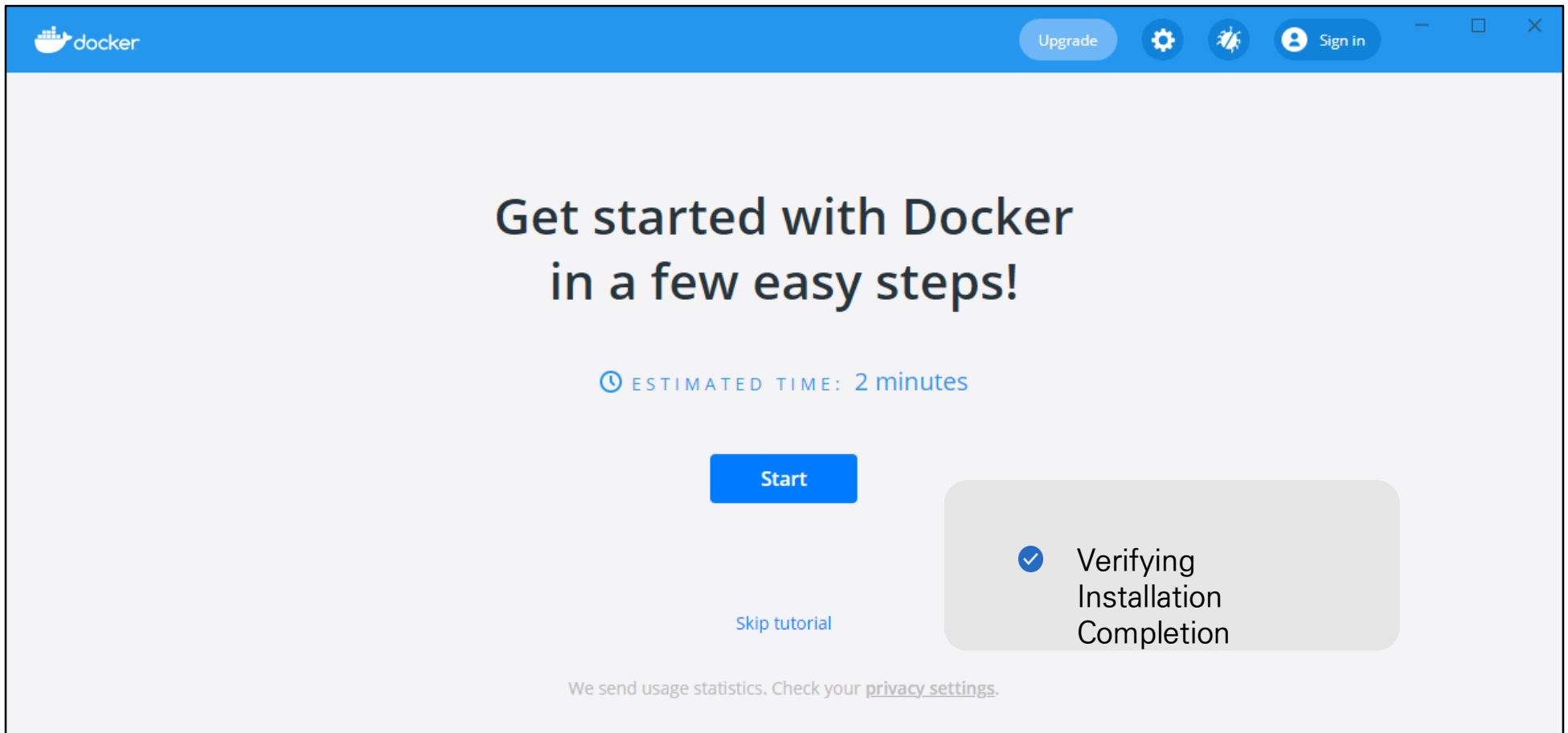
[View Full Terms](#) [Decline and Close Application](#) [Accept](#)

✓ Check I accept the terms and click Accept

01 Installing the Docker CLI

Installing the Docker CLI

Docker Desktop Installation (5/6)



01 Installing the Docker CLI

Verifying the Docker CLI Installation

Docker Desktop Installation (6/6)

```
I:\>docker version
Client:
 Cloud integration: 1.0.17
 Version:          20.10.8
 API version:      1.41
 Go version:       go1.16.6
 Git commit:       3967b7d
 Built:            Fri Jul 30 19:58:50 2021
 OS/Arch:          windows/amd64
 Context:          default
 Experimental:     true

Server: Docker Engine - Community
 Engine:
  Version:          20.10.8
  API version:      1.41 (minimum version 1.12)
  Go version:       go1.16.6
  Git commit:       75249d8
  Built:            Fri Jul 30 19:52:10 2021
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.4.9
  GitCommit:        e25210fe30a0a703442421b0f60afac609f950a3
 runc:
  Version:          1.0.1
  GitCommit:        v1.0.1-0-g4144b63
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

- ✓ Open a command prompt and enter "docker version" to verify the installation of the Docker CLI

Using Docker CLI Commands

basic form

- ✓ Docker CLI: <https://docs.docker.com/engine/reference/commandline/cli/>

Default Syntax

```
$ docker [COMMAND] [OPTIONS] [ARG...]
```

Explanation

- ✓ [COMMAND] The action you want to perform (search, pull, images, run, ps, stop, start, exec, rm, rmi, inspect, etc.)
- ✓ Details of the [OPTIONS] command (-d, -p, -v, -f, -name, --limit, etc.)
- ✓ [ARG...] Value passed to command

Using Docker CLI Commands

Image Search

Default Syntax \$ docker [COMMAND] [IMAGE] [OPTIONS]

Explanation

Search for images in the docker hub (optional --limit 6 only searches or 6)

Example of use \$ docker search nginx --limit 6

```
l:\#docker>docker search nginx --limit 6
NAME                DESCRIPTION                STARS     OFFICIAL   AUTOMATED
nginx               Official build of Nginx.    15719     [OK]
jwilder/nginx-proxy Automated Nginx reverse proxy for docker con... 2088
jlesage/nginx-proxy-manager Docker container for Nginx Proxy Manager      142     [OK]
nginxdemos/hello    NGINX webserver that serves a simple page co... 75     [OK]
nginx/nginx-ingress NGINX and NGINX Plus Ingress Controllers fo... 55
nginxinc/nginx-unprivileged Unprivileged NGINX Dockerfiles                54
```

Using Docker CLI Commands

Download Image

Default Syntax `$ docker [COMMAND] [IMAGE]:[TAG]`

Explanation Download Image from Docker Hub (Change [TAG] Partially when changing version)

Example of use `$ docker pull nginx:latest`

```
l:\#docker>docker pull nginx:latest
latest: Pulling from library/nginx
b380bbd43752: Pull complete
fca7e12d1754: Pull complete
745ab57616cb: Pull complete
a4723e260b6f: Pull complete
1c84ebdff681: Pull complete
858292fd2e56: Pull complete
Digest: sha256:644a70516a26004c97d0d85c7fe1d0c3a67ea8ab7ddf4aff193d9f301670cf36
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Using Docker CLI Commands

Image Lookup

Default Syntax	\$ docker [COMMAND]
----------------	---------------------

Explanation	View images downloaded from the docker hub locally
-------------	--

Example of use	\$ docker images
----------------	------------------

```
I:~#docker>docker images
REPOSITORY    TAG       IMAGE ID      CREATED        SIZE
nginx         latest    87a94228f133 2 weeks ago    133MB
```

Using Docker CLI Commands

Image Detail Inquiry

Default Syntax \$ docker [COMMAND] [IMAGE]

Explanation Look up images downloaded from the docker hub locally for details

Example of use \$ docker inspect nginx

```
I:\#docker>docker inspect nginx
[
  {
    "Id": "sha256:87a94228f133e2da99cb16d653cd1373c5b4e8689956386c1c12b60a20421a02",
    "RepoTags": [
      "nginx:latest"
    ],
    "RepoDigests": [
      "nginx@sha256:644a70516a26004c97d0d85c7fe1d0c3a67ea8ab7ddf4aff193d9f301670cf36"
    ],
    "Parent": "",
    "Comment": "",
    "Created": "2021-10-12T02:03:40.360294686Z",
    "Container": "21fd1c6cb532225ca7e04c77f6592e220574b919aec07021663576ef438e0fee",
```

Using Docker CLI Commands

Running a container

Default Syntax `$ docker [COMMAND] [OPTIONS] [IMAGE]`

Explanation Create and run a docker image as a container (option `-p` connects the host to the port in the container)

Example of use `$ docker run -p 80:80 nginx`

```
l:~#docker>docker run -p 80:80 nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2021/10/29 01:24:35 [notice] 1#1: using the "epoll" event method
2021/10/29 01:24:35 [notice] 1#1: nginx/1.21.3
2021/10/29 01:24:35 [notice] 1#1: built by gcc 8.3.0 (Debian 8.3.0-6)
2021/10/29 01:24:35 [notice] 1#1: OS: Linux 5.10.16.3-microsoft-standard-WSL2
2021/10/29 01:24:35 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2021/10/29 01:24:35 [notice] 1#1: start worker processes
```


Using Docker CLI Commands

Inquiry for containers

Default Syntax	\$ docker [COMMAND]
----------------	---------------------

Explanation	Check Docker Container Running
-------------	--------------------------------

Example of use	\$ docker ps
----------------	--------------

```
l:~/docker>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
f21617291c19   nginx    "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:80->80/tcp, :::80->80/tcp   exciting_feynman
```

Using Docker CLI Commands

Shutting down containers

Default Syntax	\$ docker [COMMAND] [CONTAINER_NAME]
Explanation	Shutting down a running docker container
Example of use	\$ docker stop exciting_feynman

```
l:~#docker>docker stop exciting_feynman
exciting_feynman
```

```
l:~#docker>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Using Docker CLI Commands

Running a container

Default Syntax

```
$ docker [COMMAND] [CONTAINER_NAME]
```

Explanation

Running a Docker Container that is down

Example of use

```
$ docker start exciting_feynman
```

```
l:~docker>docker start exciting_feynman
exciting_feynman
```

```
l:~docker>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
f21617291c19   nginx    "/docker-entrypoint.…"   10 minutes ago Up 2 seconds   0.0.0.0:80->80/tcp, :::80->80/tcp   exciting_feynman
```

Using Docker CLI Commands

Deleting a container

Default Syntax	<code>docker [COMMAND] [OPTIONS] [CONTAINER_NAME]</code>
----------------	--

Explanation	Deleting a Running Docker Container
-------------	-------------------------------------

Example of use	<code>docker rm -f exciting_feynman</code>
----------------	--

```
I:\#docker>docker rm -f exciting_feynman
exciting_feynman
```

```
I:\#docker>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Using Docker CLI Commands

Delete Image

Default Syntax `$ docker [COMMAND] [OPTIONS] [IMAGE_ID]`

Explanation Delete Image (option `-f` forces image deletion)

Example of use `$ docker rmi -f 87a94228f133`

```
l:~#docker>docker rmi -f 87a94228f133
Untagged: nginx:latest
Untagged: nginx@sha256:644a70516a26004c97d0d85c7fe1d0c3a67ea8ab7ddf4aff193d9f301670cf36
Deleted: sha256:87a94228f133e2da99cb16d653cd1373c5b4e8689956386c1c12b60a20421a02

l:~#docker>docker images
REPOSITORY    TAG        IMAGE ID    CREATED    SIZE
```

Using Docker CLI Commands

Login

Default Syntax	\$ docker [COMMAND] [OPTIONS] [DOCKER_HUB_ID]
----------------	---

Explanation	Docker Login
-------------	--------------

Example of use	\$ docker login -u paastakr
----------------	-----------------------------

```
l:~#docker>docker login -u paastakr
Password:
Login Succeeded
```

Using Docker CLI Commands

Image Build

Default Syntax	\$ docker [COMMAND] [OPTIONS] [IMAGE_NAME]:[TAG] [POSITION]
Explanation	Image build (optional --tag sets image name and tag)
Example of use	\$ docker build --tag edu-msa-board:latest .

```
l:~/docker#/board>docker build --tag edu-msa-board:latest .
[+] Building 3.2s (11/11) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 221B 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load metadata for docker.io/library/tomcat:9-jre8-alpine 2.6s
=> [auth] library/tomcat:pull token for registry-1.docker.io 0.0s
=> [internal] load build context 0.2s
=> => transferring context: 22.59MB 0.2s
=> [1/5] FROM docker.io/library/tomcat:9-jre8-alpine@sha256:17accf0afeeccc0310d363490cd60a788aa4630ab9c9c802231 0.0s
=> CACHED [2/5] WORKDIR /usr/local/tomcat 0.0s
=> CACHED [3/5] COPY server.xml ./conf 0.0s
=> CACHED [4/5] RUN rm -rf ./webapps/* 0.0s
```

Using Docker CLI Commands

Save Image

Default Syntax

```
$ docker [COMMAND] [OPTIONS] [FILE_NAME]  
[IMAGE_NAME]:[TAG]
```

Explanation

Save the built image (option `-o` saves the built image locally as a tar file)
Option `--tag` sets image name and tag)

Example of use

```
$ docker save -o edu-msa-board.tar edu-msa-board:latest
```

```
l:~#docker#board>docker save -o edu-msa-board.tar edu-msa-board:latest
```


Using Docker CLI Commands

Load Image

Default Syntax	\$ docker [COMMAND] [OPTIONS] [FILE_NAME]
Explanation	Load local image (option -i reads image from compressed file)
Example of use	\$ docker load -i edu-msa-board.tar

```
l:~#docker#edu-msa-board>docker load -i edu-msa-board.tar
Loaded image: edu-msa-board:latest
```

Using Docker CLI Commands

Image Tag

Default Syntax

```
$ docker [COMMAND] [BEFORE_IMAGE_NAME]  
[DOCKER_HUB_ID]/[IMAGE_NAME]
```

Explanation

Rename an existing built image to the image name that you want to upload to the image store

Example of use

```
$ docker tag edu-msa-board paastakr/edu-msa-board
```

```
l:~#docker#board>docker tag edu-msa-board paastakr/edu-msa-board
```

Using Docker CLI Commands

Upload Image

Default Syntax	\$ docker [COMMAND] [DOCKER_HUB_ID]/[IMAGE_NAME]
Explanation	Upload Image
Example of use	\$ docker push paastakr/edu-msa-board

```
I:\#docker\#board>docker push paastakr/edu-msa-board
Using default tag: latest
The push refers to repository [docker.io/paastakr/edu-msa-board]
c6ce6e8a6d88: Pushed
f5c0d88f8c41: Pushed
2a315d780e2a: Pushed
5f70bf18a086: Pushed
b7d850202de0: Pushed
d1d0b1719b96: Pushed
48988bb7b861: Pushed
edd61588d126: Pushed
9b9b7f3d56a0: Pushed
```